

RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA
RECHERCHE SCIENTIFIQUE
DIRECTION GÉNÉRALE DES ÉTUDES TECHNOLOGIQUES
INSTITUT SUPÉRIEUR DES ÉTUDES TECHNOLOGIQUES DE
RADÈS
Département Technologies de l'Informatique

Rapport de Stage de Perfectionnement

Conception et développement d'une application mobile de suivi Fitness

Réalisé par :

Mounib CHANDOUL

Encadré par :

M. Rayen BEN HASSINE

M. Amine GAMGAM

Entreprise : IT HOUSE

Année Universitaire : 2025 / 2026

Dédicaces

*C'est avec un profond respect que je dédie ce modeste travail
à*

Ma chère famille, pour leurs sacrifices, leurs précieux conseils
et leur soutien constant tout au long de mes années d'études.

*Votre présence a été pour moi un véritable réconfort, une
source de motivation, de courage et de persévérance.*

Tous mes amis, pour leur encouragement constant et leur
bienveillance. Je leur souhaite un avenir rempli de réussites.

*À tous ceux qui m'ont orienté vers la bonne voie et ont
contribué, de manière décisive, à faire de moi la personne
que je suis aujourd'hui.*

Mounib

Remerciements

C'est avec une immense reconnaissance et un réel plaisir que j'adresse mes sincères remerciements à toutes les personnes ayant, directement ou indirectement, contribué à l'accomplissement de ce travail.

*Je souhaite témoigner ma profonde reconnaissance à mon encadrant académique, **Monsieur Rayen Ben HASSINE**, pour ses recommandations pertinentes, ses conseils avisés ainsi que sa disponibilité constante.*

*Mes sincères remerciements vont également à **Monsieur Amine GAMGAM**, mon encadrant professionnel au sein de **IT HOUSE**, pour le temps généreusement accordé et la richesse des informations partagées.*

*Je remercie enfin les membres du jury ainsi que tout le corps enseignant de l'**ISET Radès**, pour l'excellence de l'enseignement qu'ils ont su m'apporter.*

Mounib

Table des matières

Dédicaces	i
Remerciements	ii
Introduction Générale	1
0.1 Problématique	2
0.2 Objectifs du projet	3
1 Étude préalable du projet	4
Sommaire du chapitre	4
1.1 Introduction	5
1.2 Cadre du projet	5
1.3 Présentation de l'organisme d'accueil	5
1.3.1 Présentation générale de IT HOUSE	5
1.3.2 Logo de IT HOUSE	6
1.3.3 Aperçu des solutions existantes	6
1.3.4 Critique de l'existant	6
1.3.5 Solution proposée	7
1.4 Méthodologie adoptée de conduite de projet	7
1.4.1 Méthodologies agiles	8
1.5 Environnement matériels et logiciels	8
1.5.1 Environnement matériels	8
1.5.2 Environnement logiciels	9
1.6 Conclusion	9
2 Mise en œuvre du projet	10
Sommaire du chapitre	10
2.1 Introduction	11
2.2 Capture des besoins	11
2.2.1 Identification des acteurs	11
2.2.2 Les besoins fonctionnels	12
2.2.3 Les besoins non fonctionnels	12
2.3 Backlog du produit	13
2.4 Diagramme du cas d'utilisation globale	15
2.5 Diagramme de classe globale	16
2.6 Architecture de l'application	17

2.6.1	Le Backend : FastAPI	17
2.6.2	Le Frontend : Flutter	17
2.7	Planification des sprints	18
2.8	Réalisation	19
2.8.1	Interface « Page d'accueil »	19
2.8.2	Interface « Administrateur »	20
2.8.3	Interface « Coach »	22
2.9	Conclusion	23
3	Sprint 1 : Gestion des utilisateurs et Authentification	24
	Sommaire du chapitre	24
3.1	Introduction	25
3.2	Backlog du sprint	25
3.3	Conception	26
3.3.1	Diagramme de cas d'utilisation	26
3.4	Diagrammes de séquence	26
3.4.1	Diagramme de séquence du CU « Inscription »	27
3.4.2	Diagramme de séquence du CU « Authentification »	28
3.5	Diagramme de classe	29
3.6	Réalisation	30
3.6.1	Interface « Inscription »	30
3.6.2	Interface « Authentification »	31
3.7	Conclusion	31
4	Sprint 2 : Gestion des comptes et Administration	32
	Sommaire du chapitre	32
4.1	Introduction	33
4.2	Backlog du sprint	33
4.3	Conception	34
4.3.1	Diagramme de cas d'utilisation	34
4.3.2	Diagramme de séquence	35
4.4	Diagramme de classe	36
4.5	Réalisation	36
4.5.1	Intégration de SQLAdmin	36
4.6	Documentation de l'API	37
4.6.1	Base URL	37
4.6.2	Authentification (JWT + bcrypt)	38
4.6.3	Gestion des utilisateurs	38
4.6.4	Suivi des coaches	39
4.6.5	Administration	39

4.6.6	Gestion des plans de fitness	40
4.6.7	Schémas de données (Pydantic)	40
4.6.8	Sécurité et dépendances	40
4.7	Conclusion	41
5	Sprint 3 : Gestion des plans de fitness et Catalogue	42
	Sommaire du chapitre	42
5.1	Introduction	43
5.2	Backlog du sprint	43
5.3	Conception	44
5.3.1	Diagramme de cas d'utilisation	44
5.3.2	Diagramme de séquence	45
5.4	Diagramme de classe	46
5.5	Réalisation	46
5.5.1	Interface liste des plans (Catalogue)	46
5.5.2	Interface de création de plan	47
5.5.3	Interface « gestion des plans »	47
5.6	Conclusion	47
	Conclusion et perspectives	48
	Bibliographie	50
	Glossaire	51
	Annexes	52
6	Code Source Principal	53
6.1	Modèle Utilisateur (Backend)	53
6.2	Endpoint d'Authentification	53
6.3	Configuration SQLAlchemy	54
6.4	Interface Utilisateur Flutter	54
7	Diagrammes Supplémentaires	56
7.1	Diagramme de Gantt	56
8	Tests et Validation	57
8.1	Tests Unitaires	57
8.2	Tests d'Interface	57

9	Guide d'Installation et Déploiement	58
9.1	Prérequis	58
9.2	Installation Backend	58
9.3	Installation Frontend	58
9.4	Configuration Production	59

Table des figures

1.1	Logo IT HOUSE	6
2.1	Diagramme de cas d'utilisation globale	15
2.2	Diagramme de classe global	16
2.3	Architechture de l'application	18
2.4	Diagramme de planification des sprints	19
2.5	Interface de la Page d'accueil.	20
2.6	Interface d'Administration et Gestion.	21
2.7	Interface d'Administration et Gestion.	22
3.1	Diagramme de cas d'utilisation du Sprint 1.	26
3.2	Diagramme de séquence de l'inscription.	27
3.3	Diagramme de séquence de l'authentification.	28
3.4	Diagramme de classes (Module Utilisateur).	29
3.5	Interface d'inscription (Vue Mobile).	30
3.6	Interface d'authentification (Vue Mobile).	31
4.1	Diagramme de cas d'utilisation du Sprint 2 (Mis à jour).	34
4.2	Diagramme de séquence : Accès au tableau de bord admin.	35
4.3	Diagramme de classes (Module Admin Étendu).	36
5.1	Diagramme de cas d'utilisation du Sprint 3.	44
5.2	Diagramme de séquence : Création d'un plan.	45
5.3	Diagramme de classes (Module Plans de Fitness).	46
7.1	Diagramme de Gantt du projet	56
7.2	Diagramme de déploiement.	56

Liste des tableaux

1.1	Caractéristiques de l'environnement matériel.	8
2.1	Le backlog du produit (Fitness Tracker)	14
3.1	Backlog du Sprint 1.	25
4.1	Backlog du Sprint 2 (Mis à jour).	33
4.2	Endpoints d'authentification	38
4.3	Endpoints de gestion des utilisateurs	38
4.4	Endpoints liés aux coaches	39
4.5	Endpoints d'administration	39
4.6	Endpoints de gestion des plans	40
5.1	Backlog du Sprint 3.	43

Listings

6.1	Modèle SQLAlchemy pour les utilisateurs	53
6.2	Route d'authentification FastAPI	53
6.3	Configuration du tableau de bord d'administration	54
6.4	Écran de connexion Flutter	54
8.1	Exemple de test FastAPI	57
9.1	Installation du backend	58
9.2	Installation du frontend	58

Introduction Générale

L'informatique occupe aujourd'hui une place essentielle dans le développement et la compétitivité des entreprises. La transformation numérique a profondément modifié les méthodes de gestion de l'information et la manière dont les services sont proposés aux utilisateurs. Dans ce contexte, les applications mobiles représentent un outil important pour offrir des solutions accessibles, interactives et adaptées aux besoins actuels.

C'est dans cette perspective que s'inscrit le présent projet, réalisé au sein du département informatique de l'entreprise IT HOUSE. Il consiste à concevoir et développer une application mobile dédiée au suivi des activités de fitness. Cette application permettra aux utilisateurs de consulter des plans d'entraînement et de suivre leur progression à travers une plateforme moderne basée sur Flutter pour le développement mobile et FastAPI pour le backend.

Ce projet vise à la fois à proposer une solution pratique pour le suivi sportif et à mettre en pratique les compétences techniques acquises durant la formation, notamment dans le développement d'applications mobiles et l'architecture logicielle.

Le présent rapport décrit les différentes étapes de conception et de réalisation de cette solution. Il est structuré en cinq chapitres :

- Chapitre 1 : Étude préalable et présentation de l'organisme d'accueil IT HOUSE.
- Chapitre 2 : Spécification des besoins et architecture du système.
- Chapitre 3 : Réalisation du Sprint 1 (authentification et gestion des comptes).
- Chapitre 4 : Réalisation du Sprint 2 (catalogue des plans de fitness).
- Chapitre 5 : Réalisation du Sprint 3 (suivi des performances et finalisation du projet).

0.1 Problématique

Le secteur du fitness digital connaît une croissance rapide, mais il est confronté à un défi technologique majeur : la **fragmentation de l'expérience utilisateur**. En analysant les solutions existantes, nous avons identifié des obstacles critiques :

- **Cloisonnement des données** : Les sportifs et les coaches utilisent souvent des outils disparates (PDF pour les programmes, réseaux sociaux pour le chat, et applications tierces pour le suivi). Cette absence de centralisation entraîne une perte d'informations et une baisse de motivation.
- **Complexité de gestion pour les coaches** : Sans plateforme dédiée, la création, l'édition et la distribution sécurisée de plans d'entraînement (contenant des exercices complexes au format JSON) restent artisanales et difficiles à gérer à grande échelle.
- **Absence de synchronisation en temps réel** : Le manque d'une architecture client-serveur robuste empêche la mise à jour instantanée des profils (bio, spécialisation) et le suivi interactif des achats de plans.

Face à ces constats, notre projet répond à la question centrale suivante : *Comment concevoir une architecture Full-Stack sécurisée capable de centraliser la création de contenu, la gestion des rôles et la communication communautaire au sein d'une interface mobile fluide et réactive ?*

Cette problématique a guidé nos choix techniques, privilégiant une séparation claire entre le frontend (Flutter) et le backend (FastAPI), avec une sécurisation via JWT et une persistance des données via SQLAlchemy. L'objectif est de fournir une expérience utilisateur cohérente, où chaque interaction (inscription, achat de plan, mise à jour de profil) est synchronisée en temps réel.

0.2 Objectifs du projet

L'objectif principal de ce projet est de concevoir et de développer une solution numérique performante, capable de centraliser l'expérience de fitness entre le coach et l'athlète. Pour répondre à la problématique identifiée, le projet s'articule autour des axes fonctionnels et techniques suivants :

- **Gestion unifiée des profils** : Mettre en place un système d'authentification et de gestion de profils distincts. L'application doit permettre aux utilisateurs de gérer leurs informations personnelles et aux coaches de présenter leur expertise, créant ainsi une base de données structurée et sécurisée pour les deux acteurs.
- **Digitalisation et suivi des plans d'entraînement** : Offrir une interface fluide permettant la consultation d'un catalogue de plans de fitness. L'objectif est de permettre à l'utilisateur de s'abonner à un programme spécifique et de suivre sa progression de manière digitale, remplaçant ainsi les supports statiques (papiers ou PDF) par une expérience interactive sur mobile.
- **Interopérabilité via une architecture API REST** : Garantir une séparation nette entre le client mobile (**Flutter**) et le serveur (**FastAPI**). L'objectif technique est d'assurer une communication fluide, rapide et sécurisée des données (utilisateurs, plans, abonnements) grâce à des services web structurés.
- **Expérience utilisateur (UI/UX) intuitive** : Développer une interface mobile moderne et réactive qui facilite la navigation, afin que l'utilisateur puisse se concentrer sur ses objectifs sportifs sans être freiné par une complexité technique.

En somme, il s'agit de livrer un produit fini qui valide la faisabilité d'un écosystème de fitness complet, allant du stockage des données en backend jusqu'à leur affichage dynamique sur smartphone. Les objectifs techniques incluent l'implémentation de fonctionnalités avancées telles que la recherche par catégories, la gestion des achats virtuels, et l'intégration d'un système de chat communautaire, tous synchronisés via une API RESTful sécurisée.

1. Étude préalable du projet

Sommaire

1.1 Introduction	5
1.2 Cadre du projet	5
1.3 Présentation de l'organisme d'accueil	5
1.4 Étude de l'existant	??
1.5 Méthodologie adoptée de conduite de projet	7
1.6 Environnement matériels et logiciels	8
1.7 Conclusion	9

1.1 Introduction

Ce premier chapitre constitue la phase d'immersion de notre projet. Il a pour objectif de poser les jalons théoriques et contextuels nécessaires à la compréhension de notre démarche de développement. Dans un premier temps, nous définirons le cadre général de ce stage de perfectionnement ainsi que les missions de notre organisme d'accueil, **IT HOUSE**, acteur clé du paysage numérique tunisien.

Par la suite, nous procéderons à une étude rigoureuse de l'existant. Cette étape est cruciale car elle permet d'identifier les lacunes des méthodes traditionnelles de suivi sportif et de justifier la pertinence de notre solution, l'application **Fitness Tracker**. Nous analyserons comment le passage d'une gestion fragmentée vers une plateforme mobile intégrée peut transformer l'interaction entre coachs et athlètes.

Enfin, nous exposerons les choix stratégiques qui ont guidé ce travail. Nous justifierons notamment l'adoption de la méthodologie **Agile**, avec un focus sur **Scrum**, pour la conduite du projet, ainsi que le choix d'une architecture découplée utilisant **Flutter** pour la réactivité de l'interface et **FastAPI** pour la performance du back-end. Cette étude préalable sert de feuille de route pour assurer la cohérence entre les besoins métiers identifiés et les solutions techniques implémentées.

1.2 Cadre du projet

Ce projet de fin de stage s'inscrit dans le cursus de la licence en Technologies de l'Informatique à l'ISET Radès. Il consiste en la conception et la réalisation d'une solution Full-Stack innovante dédiée au secteur du fitness. L'enjeu majeur est de mettre en relation directe des coachs professionnels et des clients à travers une application mobile intuitive et performante. Ce travail permet de valider l'acquisition de compétences techniques avancées, allant de la gestion de base de données asynchrone avec FastAPI jusqu'au développement d'interfaces réactives sous Flutter. Ainsi, ce stage assure la transition entre les acquis académiques et les exigences réelles du marché de l'emploi digital.

1.3 Présentation de l'organisme d'accueil

1.3.1 Présentation générale de IT HOUSE

IT HOUSE est une société de services et d'ingénierie informatique spécialisée dans le développement de solutions digitales innovantes. Elle accompagne ses clients dans leur transformation numérique en proposant des expertises en développement Web, Mobile, et Intelligence Artificielle.

- **Slogan** : DAR EL CODE.

- **Mission :** Offrir des produits technologiques de haute qualité tout en encadrant les jeunes talents.

1.3.2 Logo de IT HOUSE



Figure 1.1: Logo IT HOUSE

1.3.3 Aperçu des solutions existantes

Dans le cadre de notre étude, nous avons examiné les méthodes actuellement utilisées par les sportifs et les coachs pour la gestion des entraînements. Ces solutions peuvent être classées en deux catégories :

- **Les méthodes traditionnelles :** Utilisation de supports physiques (carnets de notes), de fichiers statiques (PDF envoyés par mail) ou de tableurs (Excel/Google Sheets). Bien que simples, ces méthodes manquent d'interactivité.
- **Les plateformes généralistes :** Utilisation de réseaux sociaux ou d'applications de messagerie (WhatsApp, Instagram) pour l'échange de conseils et de vidéos. Ces outils ne sont pas conçus pour structurer un suivi d'entraînement rigoureux.

1.3.4 Critique de l'existant

L'analyse de ces solutions met en évidence plusieurs insuffisances majeures qui freinent l'efficacité du coaching sportif moderne :

- **Fragmentation et perte d'information :** L'absence d'un outil centralisé oblige l'utilisateur à naviguer entre ses messages pour les conseils, ses fichiers pour le programme et ses notes pour le suivi. Cette dispersion nuit à l'expérience utilisateur (UX).
- **Absence de mise à jour dynamique :** Les programmes envoyés sous format PDF sont figés. Toute modification par le coach nécessite l'envoi d'un nouveau fichier, ce qui rend la gestion des versions complexe et archaïque.

- **Manque de suivi analytique** : Les solutions actuelles ne permettent pas une visualisation claire de la progression. Le sportif n'a pas de vue d'ensemble sur ses plans achetés ou sur l'historique de ses performances de manière automatisée.
- **Limites de communication** : La communication communautaire est souvent absente des applications de fitness classiques, isolant l'utilisateur dans sa pratique.

1.3.5 Solution proposée

Pour pallier ces lacunes, nous proposons le développement de l'application **Fitness Tracker**. Il s'agit d'une solution Full-Stack intégrée visant à offrir :

1. **Une centralisation complète** : Un catalogue dynamique de plans de fitness consultables et achetable directement depuis l'application mobile.
2. **Une interactivité en temps réel** : Grâce à l'architecture **FastAPI**, les modifications de profil ou de plans sont instantanément synchronisées entre le serveur et le client **Flutter**.
3. **Un aspect communautaire** : L'intégration d'un module de chat et d'un système de suivi de coachs (*Following*) pour briser l'isolement du sportif.
4. **Une gestion de rôles sécurisée** : Un accès différencié permettant aux coachs de gérer leurs contenus (CRUD) tandis que les clients se concentrent sur leur pratique.

Cette approche permet non seulement de moderniser le suivi sportif, mais aussi de garantir la pérennité et l'évolutivité du système grâce à une stack technologique performante. L'utilisation de Flutter assure une interface native sur iOS et Android, tandis que FastAPI offre une API RESTful rapide et sécurisée.

1.4 Méthodologie adoptée de conduite de projet

L'adoption d'une méthode de développement appropriée est une étape cruciale dans le cycle de vie d'un projet informatique. Pour notre projet de plateforme Fitness, nous avons choisi l'approche **Agile**, une méthodologie flexible et itérative, largement reconnue dans la gestion de projets de développement logiciel moderne.

Cette méthode permet une meilleure réactivité face aux changements, une collaboration continue avec les parties prenantes et une livraison progressive de fonctionnalités. Grâce à l'agilité, le projet peut ainsi s'adapter en fonction des retours reçus, garantissant une meilleure correspondance entre la solution Full-Stack développée et les besoins réels des utilisateurs finaux (clients et coachs).

1.4.1 Méthodologies agiles

Les méthodes agiles regroupent un ensemble de pratiques éprouvées permettant une gestion efficace des projets informatiques. Elles s'appuient sur des cycles de développement itératifs, incrémentiels et adaptatifs, offrant ainsi un pilotage plus flexible et sécurisé du projet.

Le découpage classique d'un projet (modèle en cascade) présente plusieurs limitations majeures :

- **Un délai important** entre l'expression des besoins et les premiers retours utilisateurs, ce qui peut engendrer un décalage entre le produit livré et les attentes.
- **Une faible tolérance aux erreurs**, les anomalies étant souvent détectées trop tardivement dans le cycle de développement, rendant les corrections coûteuses.
- **Une planification rigide**, avec une livraison de la valeur métier uniquement en fin de projet, ce qui réduit la flexibilité face aux imprévus techniques.

Pour répondre à ces problématiques, les méthodes agiles, et plus particulièrement la méthodologie **Scrum**, se révèlent plus adaptées. Contrairement aux approches séquentielles traditionnelles, Scrum propose un découpage du projet en **sprints** courts (généralement de 2 à 3 semaines dans notre cas), chacun se concluant par la livraison d'un incrément fonctionnel du produit (ex: module d'authentification, catalogue de plans, etc.).

Cette organisation permet d'obtenir des retours rapides, d'identifier et corriger les anomalies plus tôt, et de livrer de la valeur en continu tout au long du projet. Ce modèle permet ainsi une gestion optimisée des risques, une plus grande adaptabilité et une collaboration renforcée entre le développeur et l'organisme d'accueil **IT HOUSE**.

1.5 Environnement matériels et logiciels

1.5.1 Environnement matériels

Composant	Caractéristiques
Ordinateur	Dell G15 5535
Micro-Processeur	Intel Core i5 13450HX CPU @ up to 4.6 GHz
Mémoire RAM	32 Go
Système d'exploitation	Linux Ubuntu (64 bits)
Disque dur	512 Go NVME (Système) et 512 Go NVME (Stockage)
Carte graphique	NVIDIA GeForce RTX 3050 6G

Table 1.1: Caractéristiques de l'environnement matériel.

1.5.2 Environnement logiciels

- **VS Code** : Éditeur de code principal.
- **Flutter SDK** : Framework pour le développement mobile.
- **FastAPI** : Framework Python pour le backend.
- **PostgreSQL / SQLite** : Gestion de la base de données.
- **Postman** : Test des points de terminaison de l'API (Endpoints).
- **OverLeaf** : Rédaction du rapport.

1.6 Conclusion

Dans ce chapitre, nous avons présenté l'organisme d'accueil **IT HOUSE** et les missions qui nous ont été confiées. Nous avons détaillé le cadre du projet, les objectifs visés, ainsi que la méthodologie de développement adoptée, à savoir les méthodes agiles, avec un focus particulier sur **Scrum**.

Nous avons également exposé les ressources matérielles et les technologies choisies pour la conception et l'implémentation de la solution Full-Stack Fitness Tracker, garantissant ainsi une base technique solide pour le développement. Cette phase préparatoire nous a permis d'établir une compréhension approfondie des enjeux et de poser les fondations pour les sprints suivants, où nous implémenterons concrètement les fonctionnalités clés de l'application.

2. Mise en œuvre du projet

Sommaire

2.1 Introduction	11
2.2 Capture des besoins	11
2.3 Backlog du produit	13
2.4 Diagramme du cas d'utilisation globale	15
2.5 Diagramme de classe globale	16
2.6 Architecture de l'application	17
2.7 Planification des sprints	18
2.8 Réalisation	19
2.9 Conclusion	23

2.1 Introduction

Dans le chapitre précédent, nous avons justifié notre choix pour la méthodologie agile **Scrum**. Nous abordons à présent une étape clé de cette démarche : la phase de « Mise en œuvre du projet », souvent désignée sous le terme de « Sprint Zéro ».

Bien que ce sprint ne soit pas conçu comme un cycle de développement classique, puisqu'il ne se conclut pas par une version livrable du produit, il constitue une phase préparatoire essentielle pour structurer les bases du projet. Cette étape permet de traduire la vision métier en spécifications techniques exploitables.

Ce chapitre est donc dédié à l'identification des acteurs du système, à l'analyse rigoureuse des besoins fonctionnels et non fonctionnels, ainsi qu'à la création du **Backlog Produit**. Nous y établirons également un diagramme de cas d'utilisation global et procéderons à la planification détaillée des différents sprints. Cette organisation nous permettra d'avoir une vision claire de l'avancement technique et d'assurer la cohérence de la solution Full-Stack à venir.

2.2 Capture des besoins

En premier lieu de cette phase, il convient d'analyser le problème en prenant en compte les différentes contraintes techniques et métier, afin de garantir une solution parfaitement adaptée aux attentes des sportifs et des professionnels. Il est donc primordial d'identifier précisément les services attendus et les fonctionnalités à mettre en place pour assurer la viabilité du projet.

2.2.1 Identification des acteurs

Un acteur désigne un rôle assumé par une personne physique ou un système informatique externe interagissant avec l'application. Dans le cadre de notre plateforme de fitness, nous avons identifié deux acteurs principaux :

- **Le Client (Utilisateur)** : Il utilise l'application pour découvrir des plans d'entraînement, les acheter, suivre sa progression et interagir avec la communauté via le chat.
- **Le Coach** : Il est le créateur de contenu. Il dispose de privilèges spécifiques pour concevoir, publier et éditer des plans de fitness (via des structures JSON complexes) ainsi que pour gérer son profil professionnel (spécialisations, bio).
- **L'Administrateur** : Il est responsable de la supervision et de la gestion globale de la plateforme. Il dispose de privilèges avancés pour administrer les utilisateurs,

modérer les contenus publiés et gérer les plans de fitness. Il peut également accéder au tableau de bord administratif afin de surveiller l'activité du système et garantir le bon fonctionnement de l'application.

2.2.2 Les besoins fonctionnels

Les besoins fonctionnels définissent les services et actions que l'application doit offrir pour satisfaire les attentes des utilisateurs. Notre solution inclut les fonctionnalités suivantes :

- **Gestion de l'authentification** : L'application doit permettre la création de comptes et la connexion sécurisée via le protocole **JWT**, en distinguant les rôles dès l'inscription.
- **Gestion des profils** : Possibilité de modifier les informations personnelles, d'ajouter une biographie et une spécialisation (pour les coaches), et de mettre à jour un avatar (emoji-based).
- **Gestion des plans de fitness** :
 - *Côté Coach* : Créer, modifier et supprimer des programmes d'entraînement.
 - *Côté Client* : Rechercher par catégories, consulter les détails et effectuer un achat virtuel de plan.
- **Interaction et Suivi** : Suivre des coaches spécifiques (*Following system*) et participer à un chat communautaire pour rester motivé.

2.2.3 Les besoins non fonctionnels

Les besoins non fonctionnels correspondent aux exigences internes qui ne concernent pas directement le comportement de l'application vis-à-vis de l'utilisateur, mais qui doivent être quantifiables et restent généralement invisibles pour les utilisateurs. Les besoins non fonctionnels de notre application reposent principalement sur :

- **La performance** : L'application doit être capable de gérer un grand nombre de requêtes simultanées sans compromettre la réactivité, grâce à la nature asynchrone de **FastAPI**.
- **La Scalabilité** : L'architecture doit permettre d'évoluer facilement pour gérer une augmentation du volume d'utilisateurs ou de plans d'entraînement, notamment par le passage possible vers une base de données **PostgreSQL**.
- **La Sécurité** : Les données sensibles doivent être protégées par le hachage des mots de passe (*passlib*). L'accès aux ressources est strictement contrôlé par des jetons **JWT** selon les rôles (Client/Coach).

- **L’ergonomie** : L’interface utilisateur sous **Flutter** doit être simple, claire et intuitive, offrant une navigation fluide grâce à un design moderne (Thème sombre et accents verts).
- **L’extensibilité** : Le code doit être organisé de manière modulaire (utilisation de services et de modèles découplés) afin de faciliter l’ajout de futures fonctionnalités comme le paiement réel ou les notifications.
- **La disponibilité** : Le backend doit être structuré pour garantir un accès continu aux services API, minimisant les interruptions lors des mises à jour.

2.3 Backlog du produit

Le backlog est l’outil principal du Product Owner. Il s’agit d’une liste priorisée des tâches à réaliser par l’équipe de développement. Ce document regroupe toutes les fonctionnalités attendues du produit et est régulièrement mis à jour en fonction de l’évolution des besoins.

- **Fonctionnalité** : C’est une capacité précise attendue par le client ou les utilisateurs, répondant à un besoin spécifique.
- **User Story** : Brève description d’une fonctionnalité, rédigée du point de vue de l’utilisateur (Client ou Coach).
- **Acteurs** : Ensemble des utilisateurs qui interagissent avec la fonctionnalité.

Fonctionnalité	User Story	Acteur	Sprint
Inscription & Profil	En tant que Client ou Coach, je veux m'inscrire pour accéder aux services de la plateforme.	Client, Coach	1
Authentification	En tant qu'utilisateur, je veux m'authentifier pour sécuriser mes données et accéder à mon espace personnel.	Client, Coach	1
Gestion du Profil	En tant que Coach, je veux modifier ma bio, ma spécialisation et mon emoji pour attirer plus de clients.	Coach	1
Gestion des Plans	En tant que Coach, je veux créer, modifier et supprimer mes plans d'entraînement (exercices, prix, durée).	Coach	2
Achat & Découverte	En tant que Client, je veux parcourir le catalogue, filtrer les plans et acheter un programme de fitness.	Client	2
Interaction Sociale	En tant que Client, je veux suivre des coachs et participer au chat communautaire pour rester motivé.	Client	3

Table 2.1: Le backlog du produit (Fitness Tracker)

2.4 Diagramme du cas d'utilisation globale

Ce diagramme illustre les interactions entre les acteurs et les différentes fonctionnalités du système.

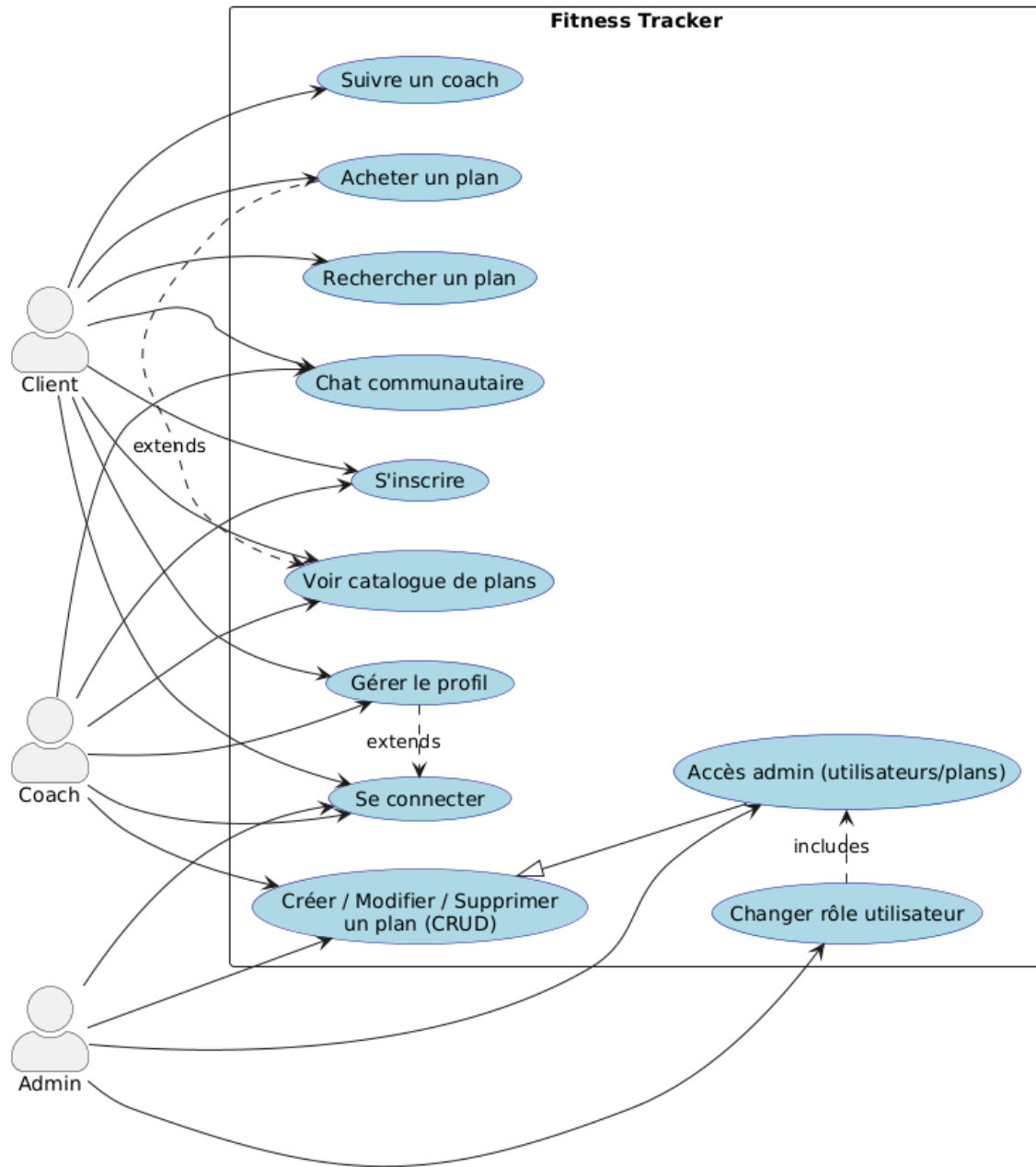


Figure 2.1: Diagramme de cas d'utilisation globale

2.5 Diagramme de classe globale

Le diagramme de classe modélise la structure de la base de données (Users, Plans, Purchases).

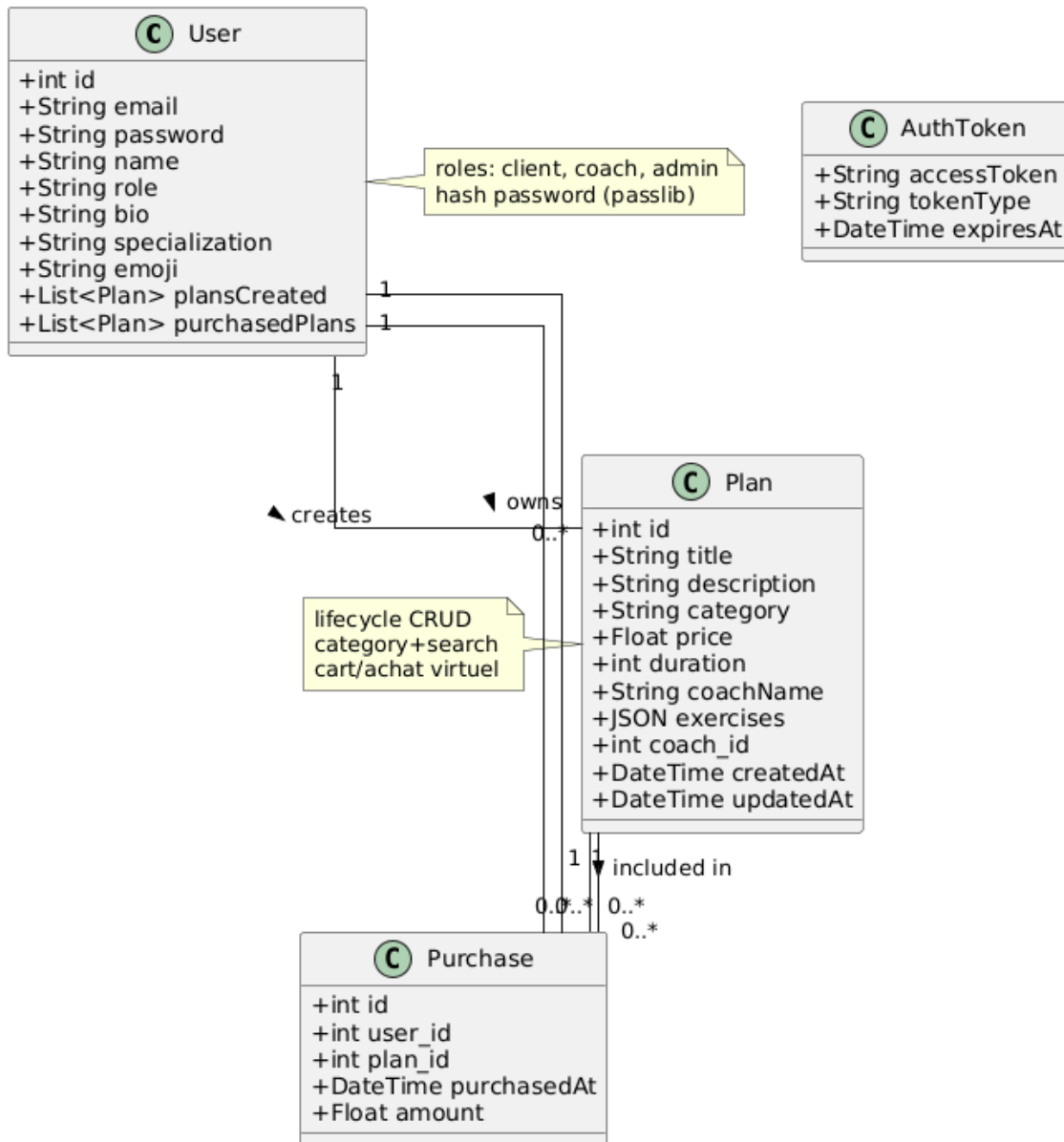


Figure 2.2: Diagramme de classe global

2.6 Architecture de l'application

Le développement de notre application repose sur une architecture **Client-Serveur** moderne, utilisant le style architectural **REST** (Representational State Transfer). Cette approche permet de séparer totalement la logique métier (Backend) de l'interface utilisateur (Frontend).

2.6.1 Le Backend : FastAPI

Pour la partie serveur, nous avons choisi **FastAPI**, un framework Web moderne et performant pour Python. Contrairement au MVC traditionnel qui gère les vues, FastAPI se concentre sur la fourniture de données structurées via des services RESTful.

- **Modèles (SQLAlchemy)** : Ils représentent la structure des données en base (Utilisateurs, Plans, Achats). Ils gèrent la logique métier et la persistance des données.
- **Schémas (Pydantic)** : Ils servent de DTO (*Data Transfer Objects*) pour valider les données entrantes et formater les réponses JSON envoyées au client.
- **Routes (Endpoints)** : Ils agissent comme des contrôleurs, recevant les requêtes HTTP de l'application mobile et appelant les fonctions nécessaires pour retourner une réponse.

2.6.2 Le Frontend : Flutter

L'interface utilisateur est développée avec **Flutter**. Ici, la séparation des responsabilités est assurée par un modèle orienté composant :

- **La Vue** : Composée de Widgets réactifs qui affichent l'interface (Catalogues, Profils, Chat).
- **La Gestion d'État (ValueNotifier)** : Assure la mise à jour fluide de l'interface dès que les données changent.
- **La Couche Service** : Gère les appels HTTP vers l'API FastAPI et la persistance locale via *SharedPreferences*.

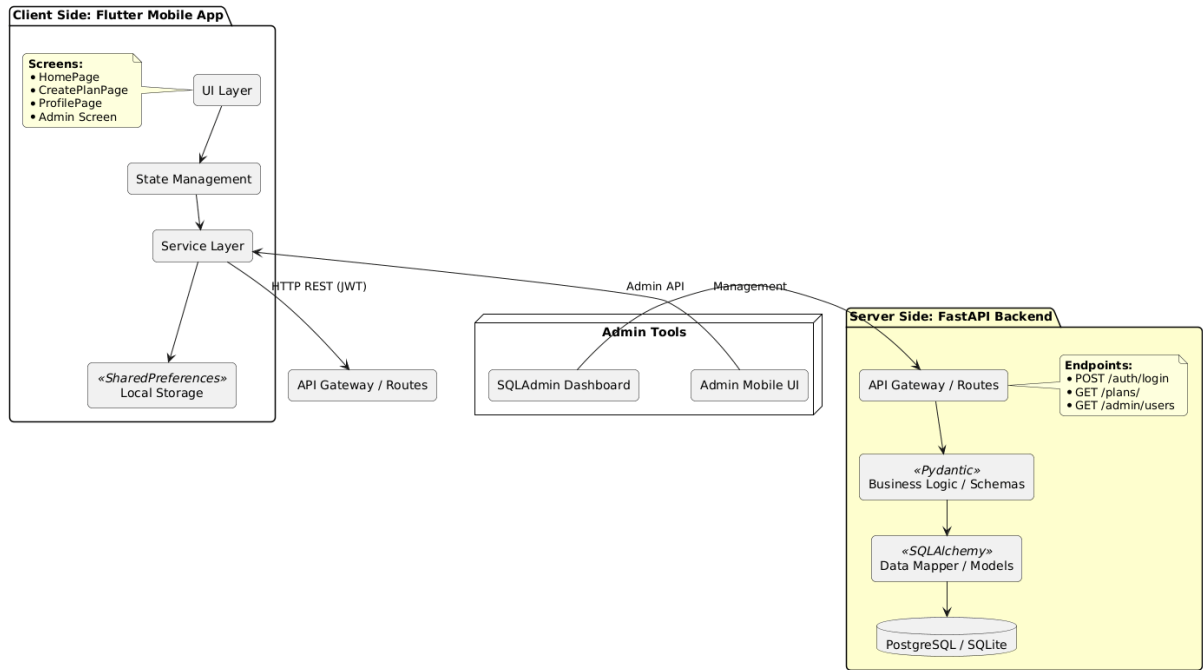


Figure 2.3: Architecture de l'application

Cette architecture assure une séparation claire des responsabilités, facilitant la maintenance, les tests et l'évolutivité du système. Par exemple, l'ajout d'une nouvelle fonctionnalité comme la recherche par catégories nécessite uniquement des modifications côté backend pour l'API et côté frontend pour l'interface, sans impact sur les autres composants.

2.7 Planification des sprints

La planification des sprints d'une release est l'événement clé dans la méthodologie Scrum. L'objectif du découpage en sprints est de préparer le planning de travail et de définir le backlog spécifique à chaque itération. Pour notre projet *Fitness Tracker*, nous avons opté pour un découpage en trois sprints majeurs, chacun visant à livrer un incrément fonctionnel testable.

- **Sprint 1 (Fondations & Utilisateurs) :** Ce sprint est dédié à la mise en place de l'environnement, au développement du backend avec FastAPI, et à l'implémentation de l'authentification sécurisée par JWT ainsi que la gestion des profils (Coach et Client).
- **Sprint 2 (Cœur de métier - Plans de Fitness) :** Il porte sur la fonctionnalité principale de l'application : la création de plans par les coaches (stockage JSON) et le catalogue de découverte permettant aux clients de parcourir et d'acheter des programmes.

- **Sprint 3 (Social & Finalisation)** : Ce dernier sprint intègre les fonctionnalités communautaires telles que le système de "Follow" entre sportifs et coaches, ainsi que le service de messagerie (Chat). Il se conclut par les phases de tests globaux et de préparation au déploiement.

La figure ci-dessous décrit la structuration chronologique de ces sprints :

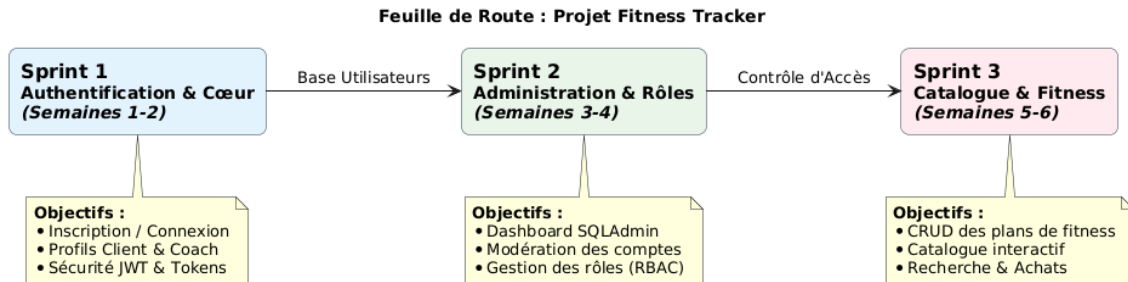


Figure 2.4: Diagramme de planification des sprints

Chaque sprint a été calibré pour durer environ deux semaines, permettant ainsi une évaluation régulière de l'avancement et une adaptation rapide aux défis techniques rencontrés lors de l'intégration entre Flutter et FastAPI.

2.8 Réalisation

Dans cette section, nous présentons les interfaces clés issues du processus de développement. Ces captures d'écran illustrent l'ergonomie de l'application **Fitness Tracker** et la mise en œuvre des fonctionnalités décrites précédemment.

2.8.1 Interface « Page d'accueil »

La page d'accueil constitue le point d'entrée principal pour l'utilisateur (Client). Elle affiche un aperçu dynamique des plans de fitness disponibles, les nouveautés des coaches suivis, et un accès rapide aux dernières séances d'entraînement.

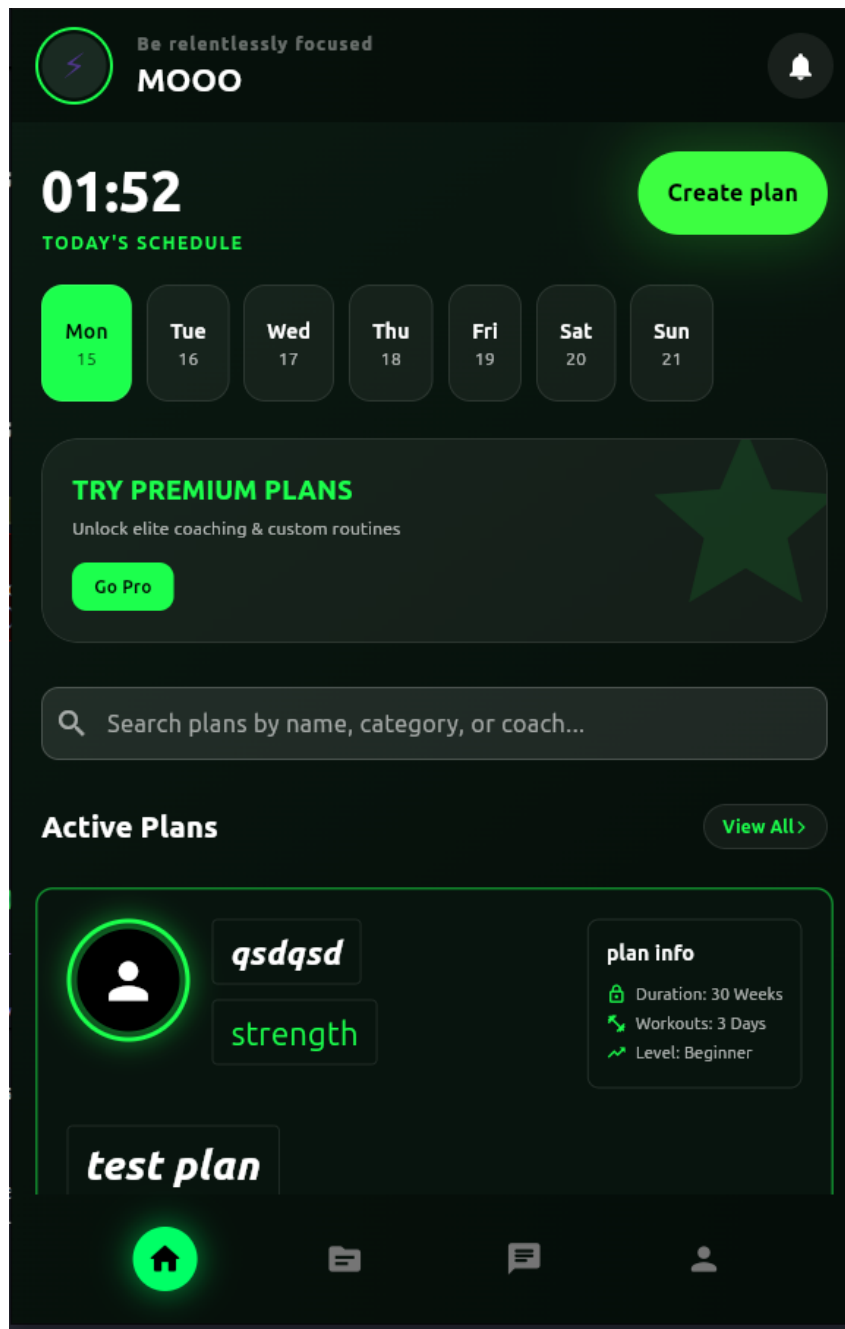


Figure 2.5: Interface de la Page d'accueil.

2.8.2 Interface « Administrateur »

L'interface administrateur est conçue pour la supervision de la plateforme. Elle permet une gestion centralisée des utilisateurs (Clients et Coaches). Ses fonctionnalités incluent la visualisation des statistiques globales, la modération des contenus publiés et la gestion des droits d'accès.

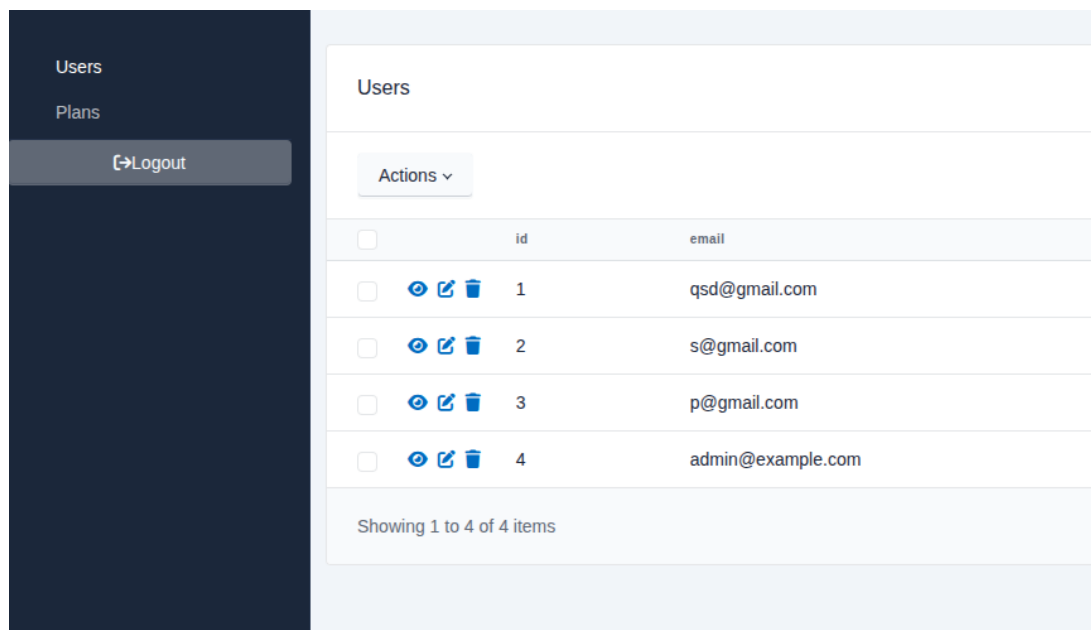


Figure 2.6: Interface d'Administration et Gestion.

2.8.3 Interface « Coach »

Cette interface est réservée aux professionnels du fitness. Elle propose des outils spécifiques pour la création de programmes, l'édition de la biographie professionnelle et le suivi des statistiques de vente des plans d'entraînement.

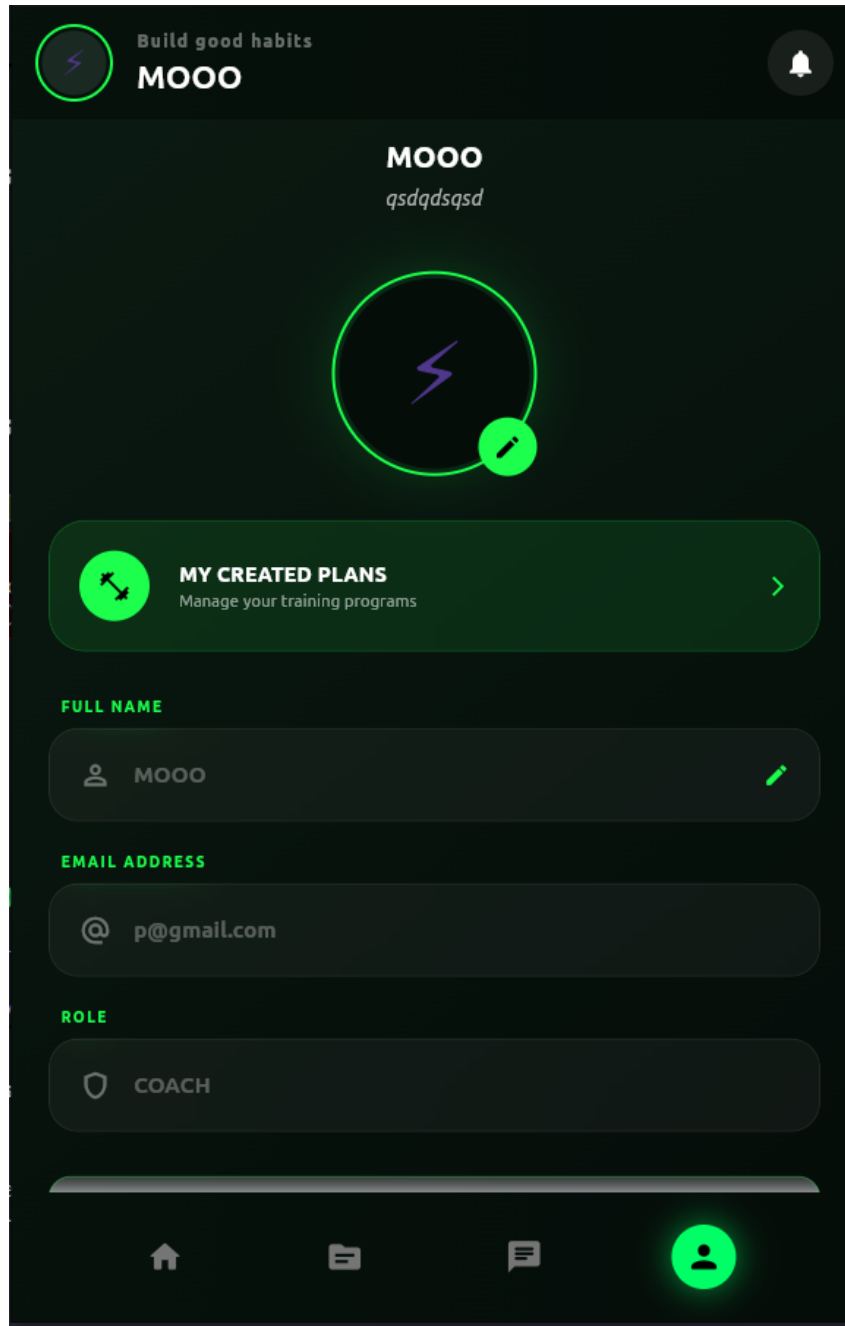


Figure 2.7: Interface d'Administration et Gestion.

2.9 Conclusion

Dans ce chapitre, nous avons présenté les différents acteurs du système ainsi que les besoins fonctionnels et non fonctionnels liés à notre projet. Nous avons également exposé les principales fonctionnalités prévues, la conception générale de l'application, son architecture découplée **Flutter/FastAPI**, le backlog produit et la planification des sprints.

Cette phase de spécification et de conception logicielle nous a permis d'établir une feuille de route claire et structurée. Le chapitre suivant sera dédié à une description détaillée du travail effectué lors du **Premier Sprint** de notre projet, portant sur la mise en place de l'authentification et de la gestion des profils.

3. Sprint 1 : Gestion des utilisateurs et Authentification

Sommaire

3.1 Introduction	25
3.2 Backlog du sprint	25
3.3 Conception	26
3.4 Diagrammes de séquence	26
3.5 Diagramme de classe	29
3.6 Réalisation	30
3.7 Conclusion	31

3.1 Introduction

Ce troisième chapitre est dédié au premier cycle de développement de notre projet, le **Sprint 1**. L'objectif principal de cette itération est de mettre en place le système d'authentification et de gestion des profils. Cette étape est cruciale car elle définit la structure de sécurité de l'application via les jetons **JWT** et permet de distinguer les deux types d'acteurs : le *Client* et le *Coach*.

Durant ce sprint, nous avons implémenté les endpoints FastAPI pour l'inscription et la connexion, intégré la validation des données avec Pydantic, et développé les interfaces Flutter pour la gestion des comptes. Cette phase a posé les bases de la sécurité et de l'expérience utilisateur, avec une attention particulière à la persistance des sessions via SharedPreferences.

3.2 Backlog du sprint

Le backlog du Sprint 1 définit les tâches prioritaires à réaliser durant cette période pour fournir un incrément fonctionnel sécurisé.

ID	User Story	Priorité
US1.1	En tant qu'utilisateur, je veux m'inscrire pour créer un compte.	Haute
US1.2	En tant qu'utilisateur, je veux m'authentifier pour accéder à mon espace.	Haute
US1.3	En tant que Coach, je veux configurer ma bio et ma spécialisation.	Haute
US1.4	En tant qu'utilisateur, je veux pouvoir me déconnecter en toute sécurité.	Moyenne

Table 3.1: Backlog du Sprint 1.

3.3 Conception

3.3.1 Diagramme de cas d'utilisation

Le diagramme suivant détaille les interactions spécifiques aux fonctionnalités de ce sprint.

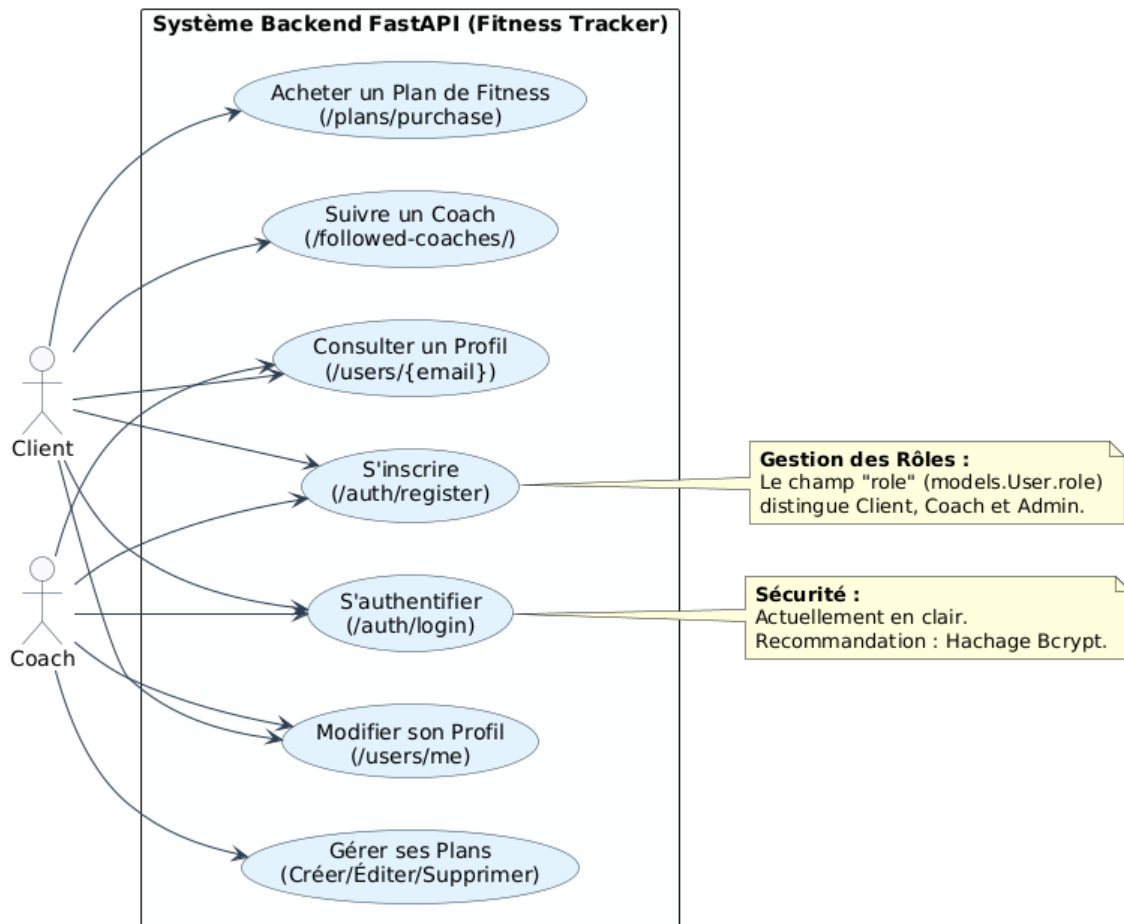


Figure 3.1: Diagramme de cas d'utilisation du Sprint 1.

3.4 Diagrammes de séquence

Les diagrammes de séquence illustrent les échanges entre le client Flutter, le serveur FastAPI et la base de données.

3.4.1 Diagramme de séquence du CU « Inscription »

Ce diagramme montre le processus de création d'un compte avec le hachage du mot de passe côté serveur.

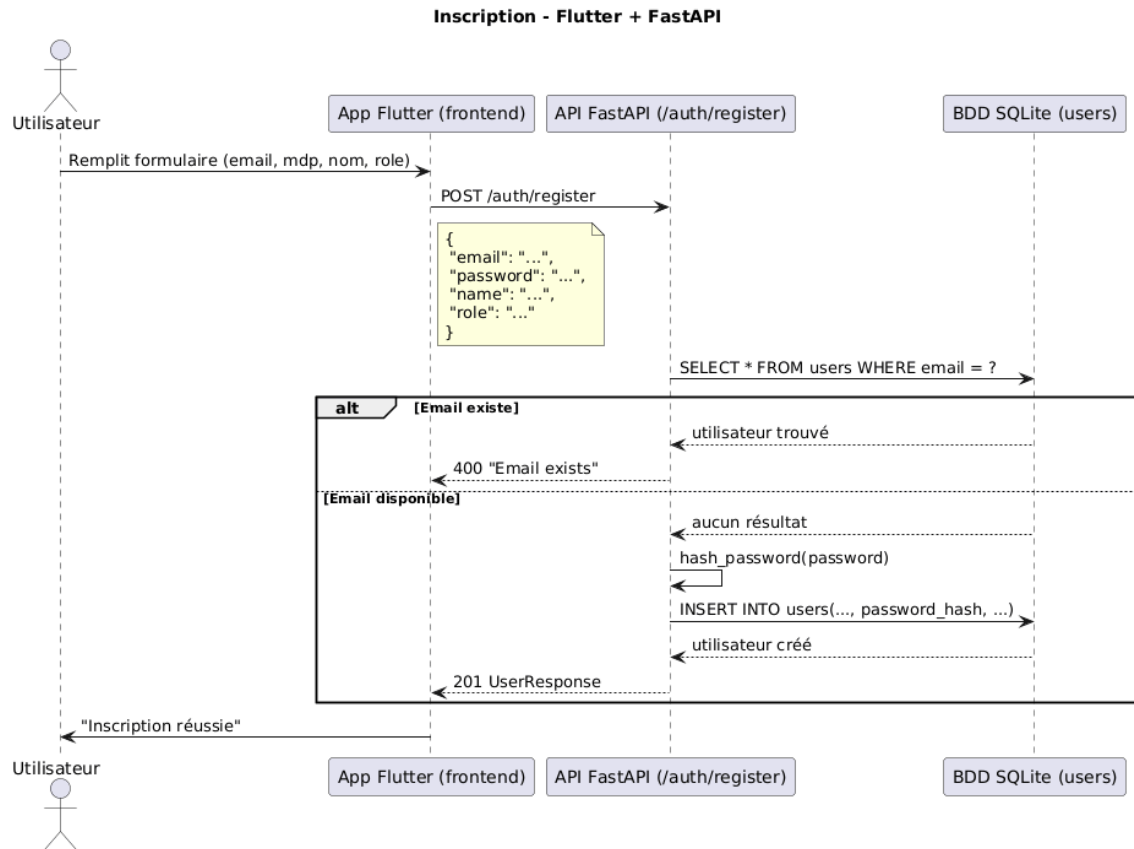


Figure 3.2: Diagramme de séquence de l'inscription.

3.4.2 Diagramme de séquence du CU « Authentification »

Il illustre la vérification des identifiants et la génération du jeton **JWT** (JSON Web Token).

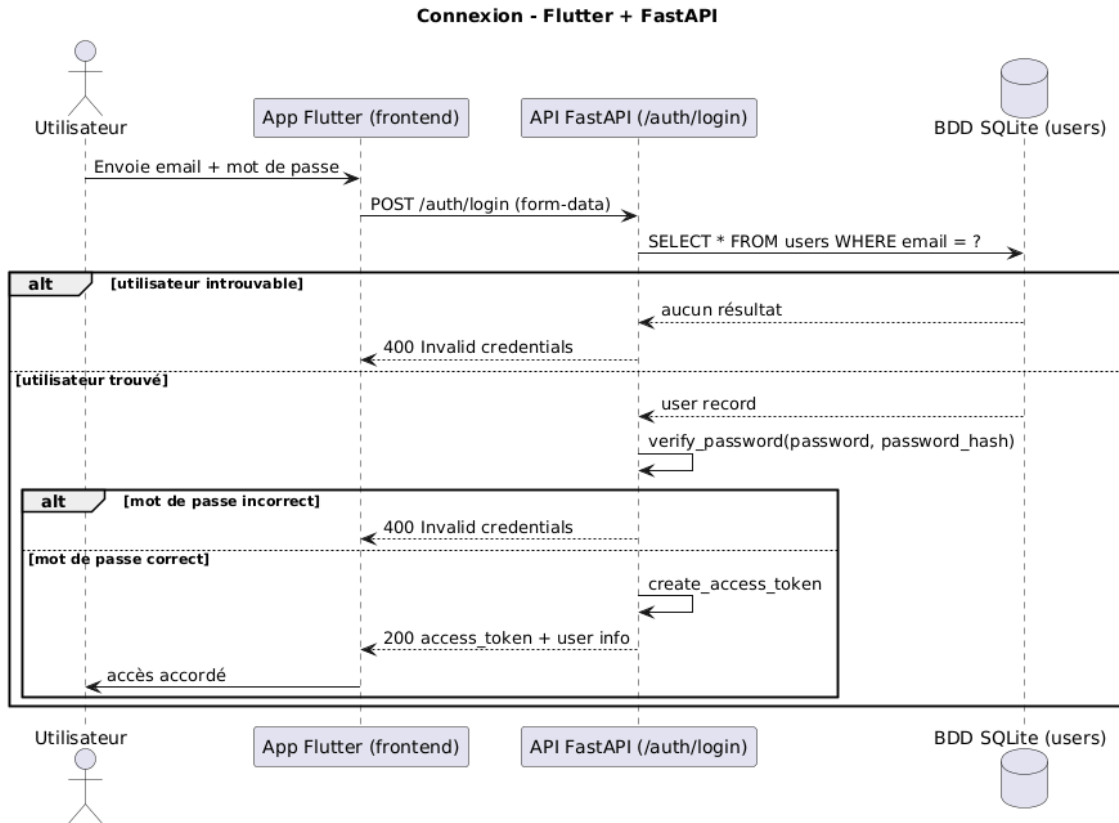


Figure 3.3: Diagramme de séquence de l'authentification.

3.5 Diagramme de classe

Ce diagramme représente la structure de la table *User* incluant les attributs tels que le rôle, la bio et la spécialisation.

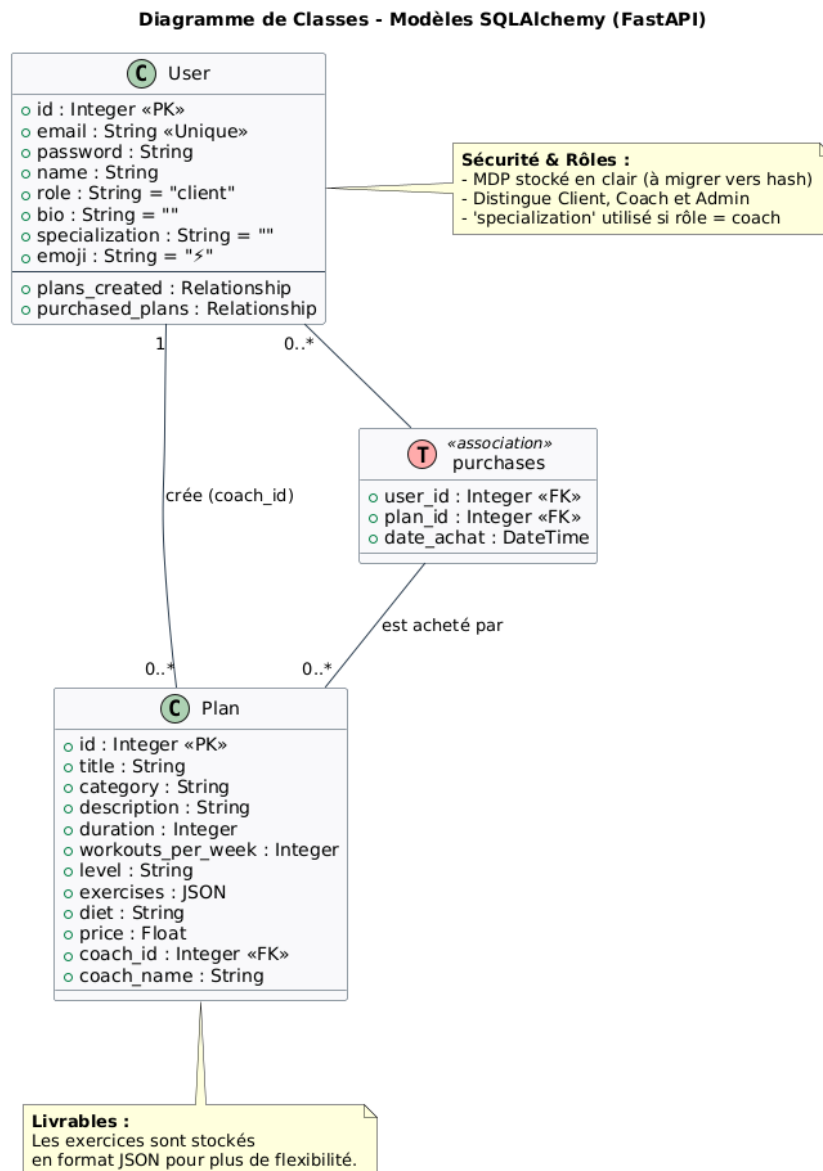
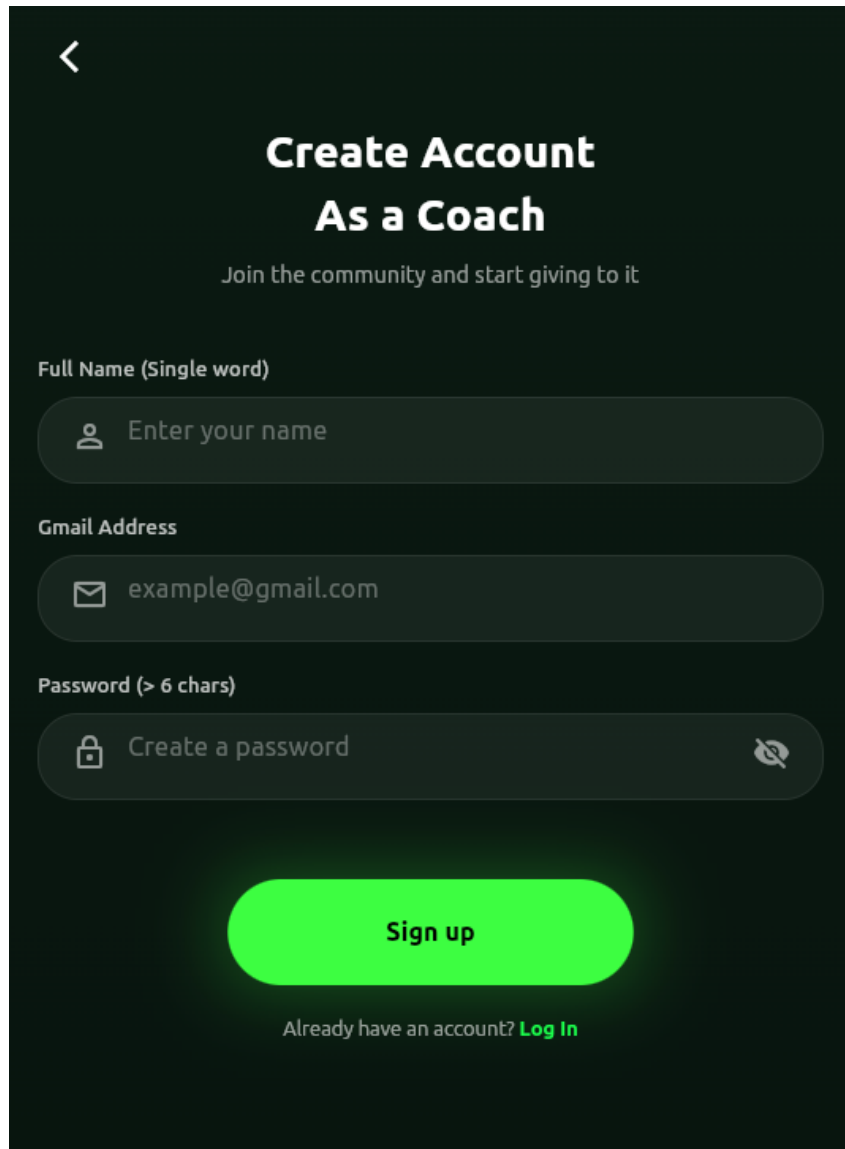


Figure 3.4: Diagramme de classes (Module Utilisateur).

3.6 Réalisation

3.6.1 Interface « Inscription »

L'interface d'inscription permet à l'utilisateur de choisir son rôle et de remplir ses informations de base avec une validation de formulaire en temps réel sous Flutter.



The screenshot displays a mobile application interface for creating a coach account. At the top left is a back arrow icon. The title 'Create Account As a Coach' is centered in a large, bold, black font. Below the title is the subtitle 'Join the community and start giving to it' in a smaller, gray font. The form consists of three input fields, each with a label and an icon: 'Full Name (Single word)' with a person icon, 'Gmail Address' with an envelope icon, and 'Password (> 6 chars)' with a lock icon. The first two fields contain placeholder text 'Enter your name' and 'example@gmail.com' respectively. The password field contains the text 'Create a password' and has a toggle icon on the right. Below the form is a large, rounded, orange button labeled 'Sign up'. At the bottom, there is a link that says 'Already have an account? Log In'.

Figure 3.5: Interface d'inscription (Vue Mobile).

3.6.2 Interface « Authentication »

La page de connexion utilise une architecture réactive pour gérer les états de chargement et stocker le jeton localement via *SharedPreferences*.

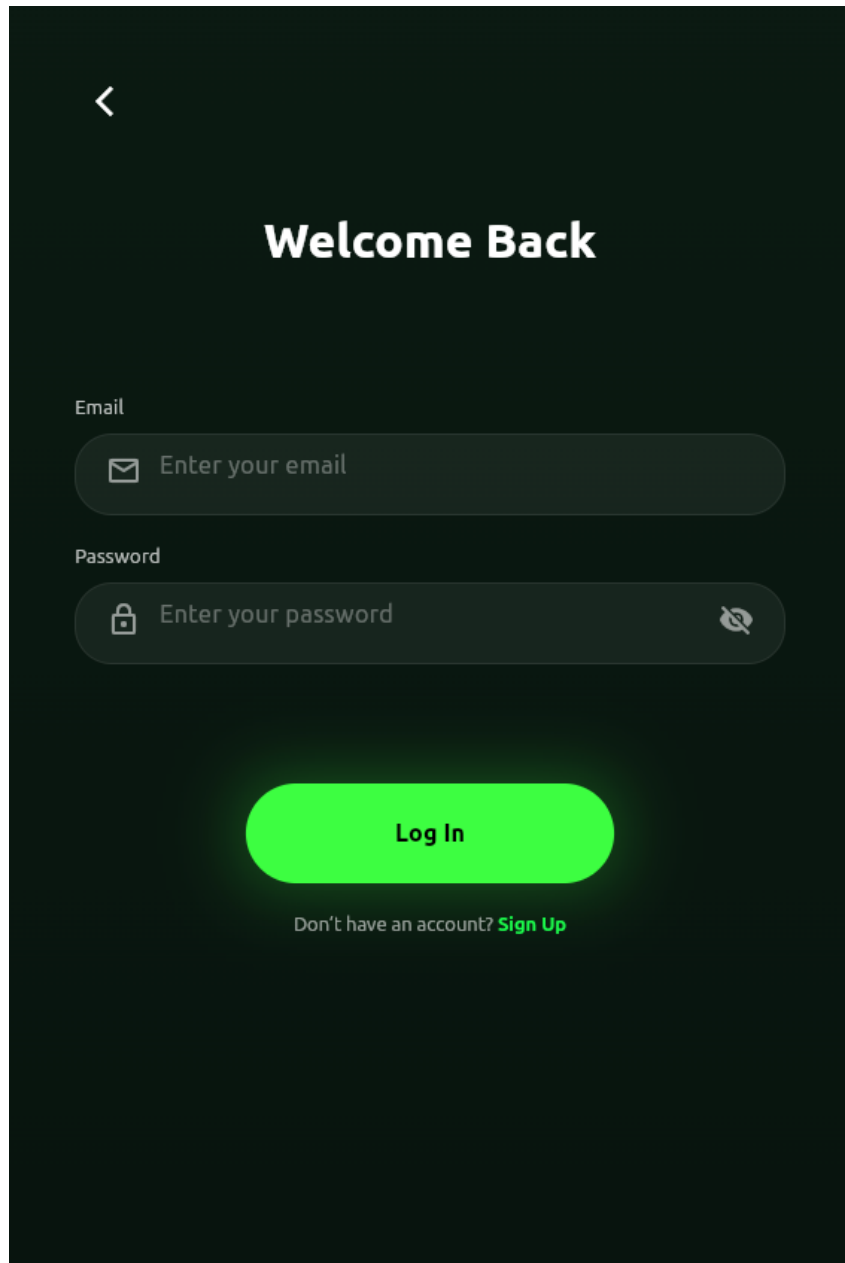


Figure 3.6: Interface d'authentification (Vue Mobile).

3.7 Conclusion

Ce premier sprint nous a permis de poser les bases de la sécurité et de la gestion des identités dans notre application. L'intégration entre le frontend Flutter et le backend FastAPI est opérationnelle pour la gestion des comptes. Le chapitre suivant portera sur le **Sprint 2**, dédié à la création et à la gestion des plans de fitness.

4. Sprint 2 : Gestion des comptes et Administration

Sommaire

4.1 Introduction	33
4.2 Backlog du sprint	33
4.3 Conception	34
4.4 Diagramme de classe	36
4.5 Réalisation	36
4.6 Conclusion	41

4.1 Introduction

Après avoir mis en place l'authentification lors du premier sprint, ce deuxième cycle de développement, le **Sprint 2**, se concentre sur les fonctionnalités d'administration avancées et la personnalisation des profils. L'objectif est de permettre une gestion rigoureuse des utilisateurs de la plateforme (Clients, Coachs et Administrateurs) et d'offrir des outils puissants pour la maintenance du système.

Durant cette itération, nous avons intégré **SQLAdmin**, un tableau de bord web pour l'administration de la base de données, et développé une interface mobile pour la gestion des utilisateurs. Ces fonctionnalités permettent aux administrateurs de gérer efficacement les comptes utilisateurs, les plans de fitness, et d'assurer la sécurité de la plateforme.

4.2 Backlog du sprint

Ce backlog détaille les récits utilisateurs (User Stories) liés à l'administration avancée du système.

ID	User Story	Priorité
US2.1	En tant qu'administrateur, je veux accéder à un tableau de bord web pour visualiser et gérer tous les utilisateurs et plans.	Haute
US2.2	En tant qu'administrateur, je veux pouvoir modifier les rôles des utilisateurs (client, coach, admin).	Haute
US2.3	En tant qu'administrateur, je veux supprimer des comptes utilisateurs problématiques.	Moyenne
US2.4	En tant qu'administrateur, je veux une interface mobile pour gérer les utilisateurs depuis l'application.	Moyenne
US2.5	En tant qu'utilisateur, je veux consulter et mettre à jour mes informations de profil (bio, spécialité).	Haute

Table 4.1: Backlog du Sprint 2 (Mis à jour).

4.3 Conception

4.3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation pour ce sprint met l'accent sur les privilèges administratifs étendus et les nouvelles fonctionnalités de gestion.

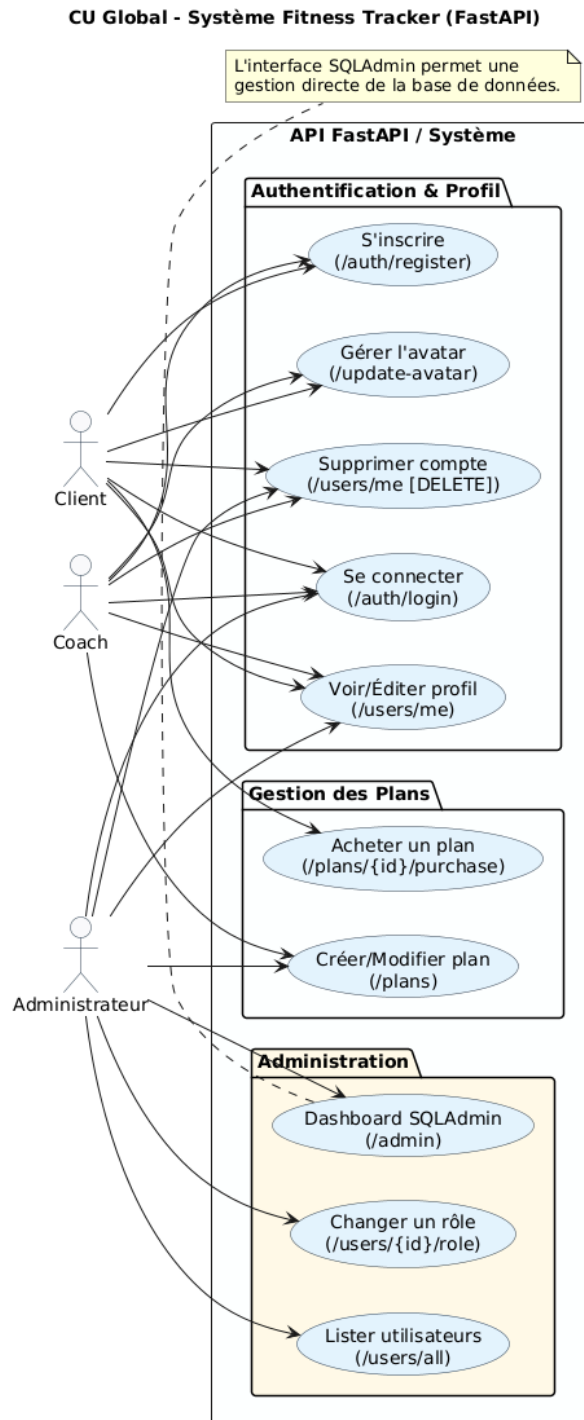


Figure 4.1: Diagramme de cas d'utilisation du Sprint 2 (Mis à jour).

4.3.2 Diagramme de séquence

Le diagramme suivant illustre le processus d'administration via SQLAdmin, de l'authentification à la modification des données.

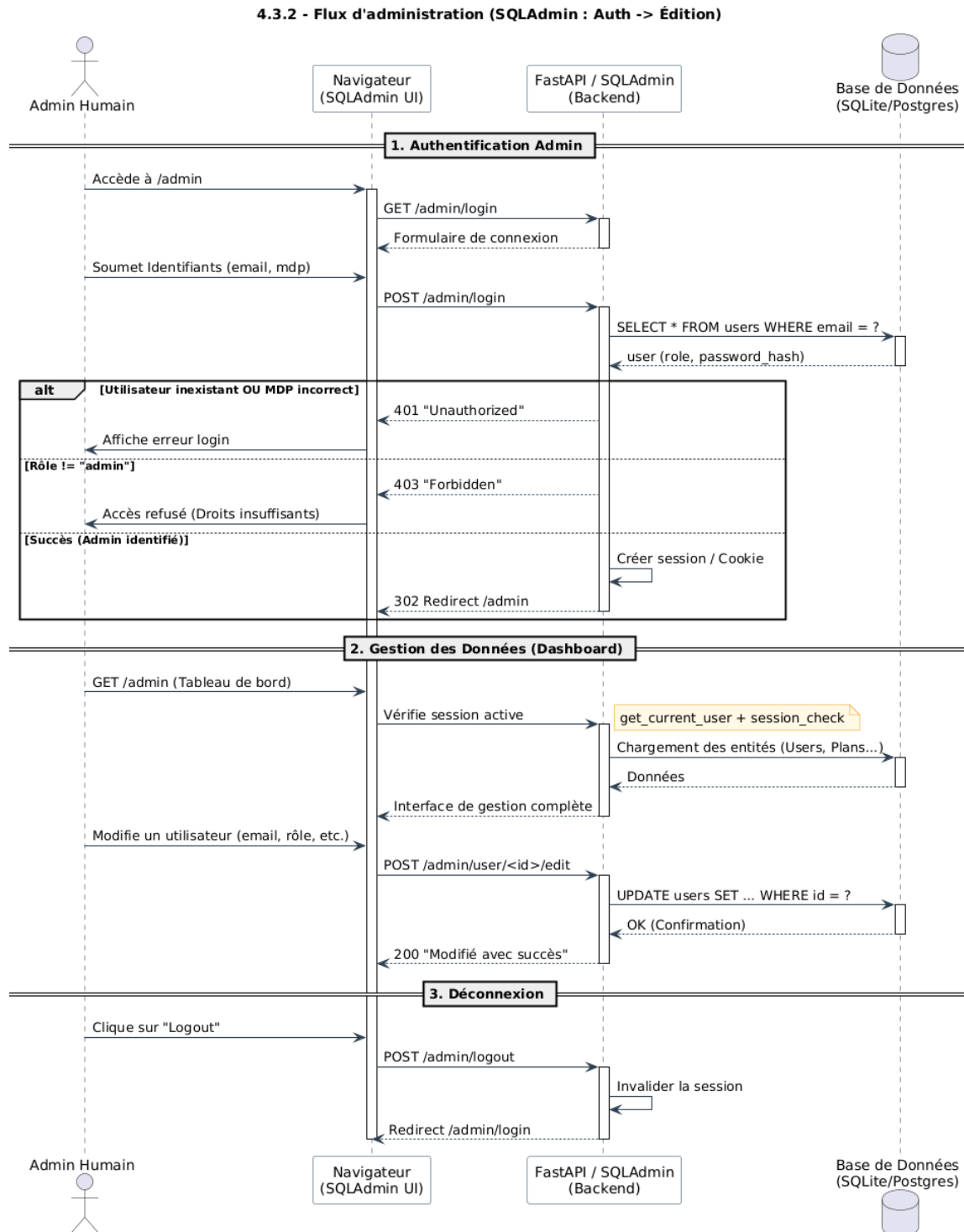


Figure 4.2: Diagramme de séquence : Accès au tableau de bord admin.

4.4 Diagramme de classe

Cette section présente l'évolution du schéma de données avec l'ajout des fonctionnalités d'administration.

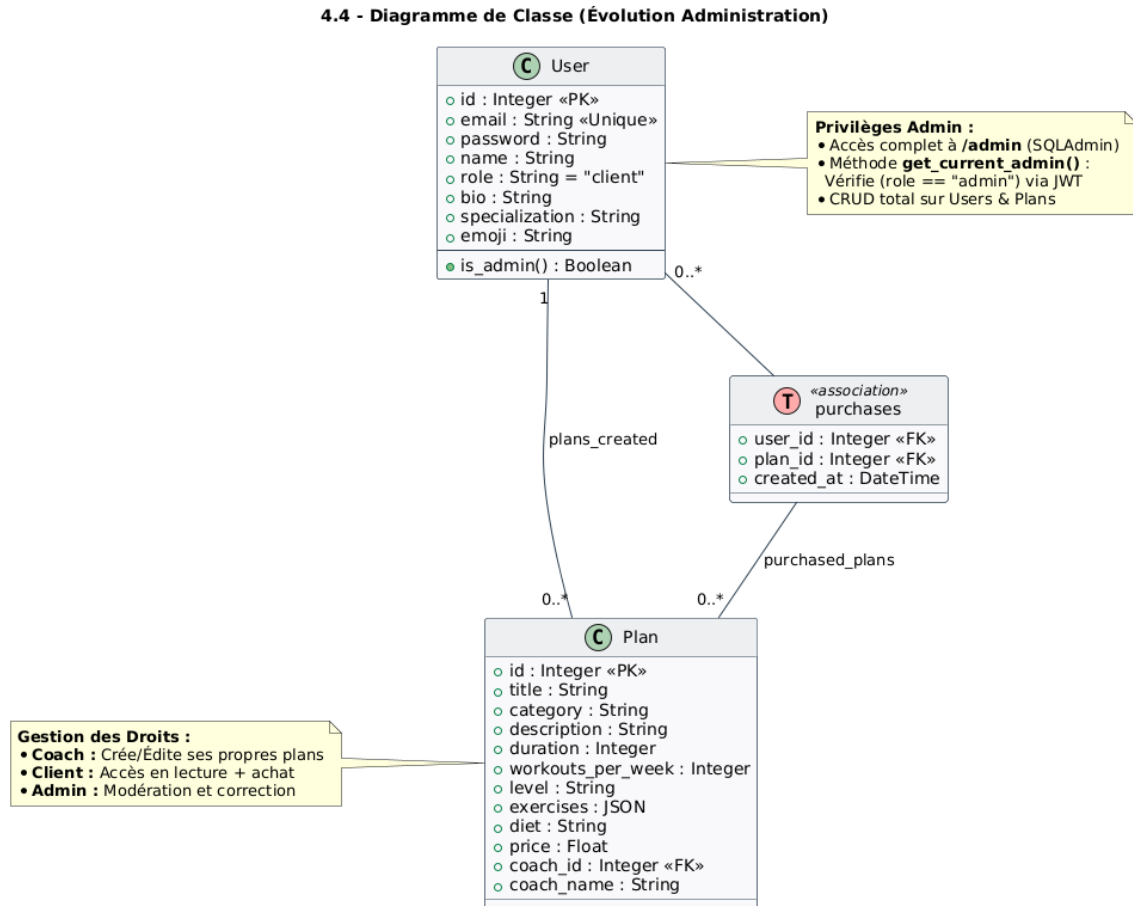


Figure 4.3: Diagramme de classes (Module Admin Étendu).

4.5 Réalisation

L'implémentation de ce sprint a intégré SQLAdmin et développé des interfaces de gestion avancées.

4.5.1 Intégration de SQLAdmin

Nous avons intégré **SQLAdmin**, une bibliothèque Python qui fournit une interface web automatique pour l'administration de bases de données SQLAlchemy. Cette solution offre :

- Une interface web responsive accessible via `/admin`
- Authentification sécurisée avec vérification des rôles

- Gestion CRUD complète pour les modèles User et Plan
- Recherche et filtrage avancés
- Pagination automatique

Le code d'intégration comprend :

```

1 from sqladmin import Admin, ModelView
2 from sqladmin.authentication import AuthenticationBackend
3
4 class AdminAuth(AuthenticationBackend):
5     async def login(self, request: Request) -> bool:
6         # V r i f i c a t i o n   d e s   c r e d e n t i a l s   e t   d u   r   l e   a d m i n
7         # ...
8
9     async def authenticate(self, request: Request) -> bool:
10         return "user" in request.session
11
12 admin = Admin(api, models.engine, authentication_backend=AdminAuth(
13     secret_key="..."))
14
15 class UserAdmin(ModelView, model=models.User):
16     column_list = [models.User.id, models.User.email, models.User.name,
17         models.User.role]
18
19 class PlanAdmin(ModelView, model=models.Plan):
20     column_list = [models.Plan.id, models.Plan.title, models.Plan.
21         category, models.Plan.coach_name, models.Plan.price]
22
23 admin.add_view(UserAdmin)
24 admin.add_view(PlanAdmin)

```

4.6 Documentation de l'API

Cette section décrit les principales routes de l'API REST développée avec **FastAPI**. L'API est utilisée par l'application mobile Flutter pour communiquer avec le serveur backend.

4.6.1 Base URL

- Environnement local (développement) : <http://localhost:8000>
- Documentation Swagger : <http://localhost:8000/docs>
- Documentation ReDoc : <http://localhost:8000/redoc>

4.6.2 Authentification (JWT + bcrypt)

Endpoint	Méthode	Description
/auth/register	POST	Création d'un utilisateur. Reçoit <code>email</code> , <code>password</code> , <code>name</code> , <code>role</code> . Le mot de passe est hashé avec <code>bcrypt</code> avant stockage.
/auth/login	POST	Authentification via OAuth2 form-data (<code>username</code> , <code>password</code>). Retourne un <code>access_token</code> JWT ainsi que les informations utilisateur.

Table 4.2: Endpoints d'authentification

4.6.3 Gestion des utilisateurs

Endpoint	Méthode	Description
/users/all	GET	Retourne la liste complète des utilisateurs (<code>List [UserResponse]</code>).
/users/{user_gmail}	GET	Recherche un utilisateur par email. Retourne une erreur 404 si l'utilisateur n'existe pas.
/users/me	PATCH	Met à jour les informations du profil de l'utilisateur connecté (<code>UserUpdate</code>). Authentification requise.
/update-profile	PATCH	Alias de <code>/users/me</code> utilisé par l'application Flutter pour modifier le profil.
/update-avatar	PATCH	Met à jour l'emoji de l'utilisateur (exemple : <code>{"emoji":""}</code>).
/users/me	DELETE	Supprime le compte de l'utilisateur connecté.

Table 4.3: Endpoints de gestion des utilisateurs

4.6.4 Suivi des coaches

Endpoint	Méthode	Description
/followed-coaches/{user_email}	GET	Retourne la liste des coaches suivis par l'utilisateur avec leurs informations (id, name, specialization, emoji) et les plans associés. Authentification requise.
/coach-profile	PATCH	Permet à un utilisateur ayant le rôle coach de modifier son profil professionnel (bio, spécialisation). Retourne 403 si le rôle n'est pas coach.

Table 4.4: Endpoints liés aux coaches

4.6.5 Administration

Endpoint	Méthode	Description
/admin/users	GET	Retourne la liste de tous les utilisateurs. Accès réservé aux administrateurs.
/admin/users/{user_id}	DELETE	Supprime un utilisateur par identifiant. Retourne 404 si l'utilisateur est introuvable.
/admin/users/{user_id}/role	PATCH	Modifie le rôle d'un utilisateur (RoleUpdate : {"new_role": "coach"}).

Table 4.5: Endpoints d'administration

4.6.6 Gestion des plans de fitness

Endpoint	Méthode	Description
/plans/	POST	Création d'un plan de fitness par un coach. Reçoit <code>title</code> , <code>category</code> , <code>duration</code> , <code>workouts_per_week</code> , <code>level</code> , <code>exercises</code> , <code>price</code> , <code>description</code> .
/plans/	GET	Retourne la liste de tous les plans disponibles (<code>List[PlanResponse]</code>).
/plans/my	GET	Retourne les plans créés par l'utilisateur connecté.
/plans/{plan_id}	DELETE	Supprime un plan si l'utilisateur connecté est le coach propriétaire.
/plans/{plan_id}/purchase	POST	Permet à un utilisateur d'acheter un plan de fitness. Retourne un message de succès ou indique si le plan est déjà possédé.

Table 4.6: Endpoints de gestion des plans

4.6.7 Schémas de données (Pydantic)

Les principaux modèles de données utilisés dans l'API sont :

- `UserCreate`, `UserUpdate`, `UserResponse`
- `RoleUpdate`
- `PlanBase`, `PlanCreate`, `PlanResponse`

Ces schémas assurent la validation des données entrantes et la structure des réponses JSON envoyées au client.

4.6.8 Sécurité et dépendances

L'API utilise plusieurs dépendances pour garantir la sécurité et l'accès aux ressources :

- `get_db` : fournit une session de base de données SQLAlchemy.
- `get_current_user` : vérifie et décode le token JWT pour authentifier l'utilisateur.
- `get_current_admin` : vérifie que l'utilisateur possède le rôle `admin`.

4.7 Conclusion

Le Sprint 2 a considérablement renforcé les capacités d'administration de la plateforme en intégrant des outils professionnels comme SQLAdmin et en développant des interfaces mobiles dédiées. Ces fonctionnalités assurent une gestion efficace et sécurisée des utilisateurs et des contenus. Le chapitre suivant portera sur le **Sprint 3**, dédié à la gestion complète des plans de fitness et du catalogue.

5. Sprint 3 : Gestion des plans de fitness et Catalogue

Sommaire

5.1 Introduction	43
5.2 Backlog du sprint	43
5.3 Conception	44
5.4 Diagramme de classe	46
5.5 Réalisation	46
5.6 Conclusion	47

5.1 Introduction

Ce cinquième chapitre présente le **Sprint 3**, l'itération la plus dense techniquement. Après avoir sécurisé l'accès et géré les profils, nous implémentons ici le cœur métier de la plateforme : la gestion des programmes d'entraînement. Ce cycle englobe la création de contenus complexes par les coachs et la mise en place d'un catalogue dynamique permettant aux clients de découvrir et de consulter les exercices.

Durant ce sprint, nous avons finalisé l'intégration complète entre Flutter et FastAPI, implémenté la persistance des données avec SQLAlchemy, et développé un système de recherche avancé. Cette phase a transformé notre prototype en une application fonctionnelle, avec des tests d'intégration et une optimisation des performances pour garantir une expérience utilisateur fluide.

5.2 Backlog du sprint

Le backlog de ce sprint se concentre sur les opérations CRUD (Create, Read, Update, Delete) des plans de fitness.

ID	User Story	Priorité
US3.1	En tant que Coach, je veux créer un plan avec des exercices, un prix et une catégorie.	Haute
US3.2	En tant que Client, je veux parcourir la liste des plans disponibles par catégorie.	Haute
US3.3	En tant que Coach, je veux modifier ou supprimer mes propres plans.	Haute
US3.4	En tant que Client, je veux rechercher un plan spécifique par mots-clés.	Moyenne

Table 5.1: Backlog du Sprint 3.

5.3 Conception

5.3.1 Diagramme de cas d'utilisation

Ce diagramme illustre les interactions liées à la gestion du catalogue. Le coach est l'initiateur de la création, tandis que le client interagit principalement avec la consultation.

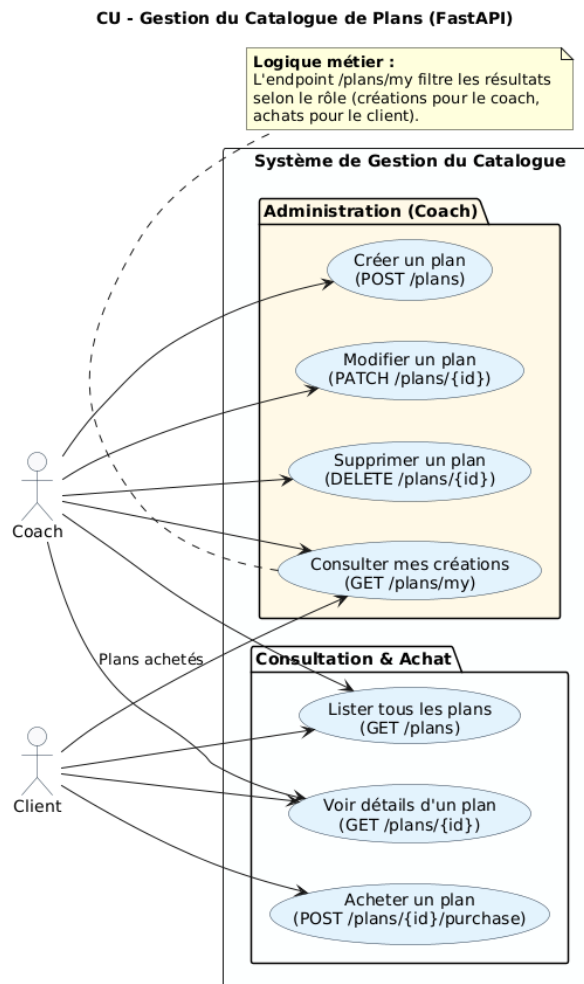


Figure 5.1: Diagramme de cas d'utilisation du Sprint 3.

5.3.2 Diagramme de séquence

Nous détaillons ici le processus de création d'un plan de fitness. Le défi technique réside dans la transmission d'une liste d'exercices structurée au format JSON du client Flutter vers l'API FastAPI.

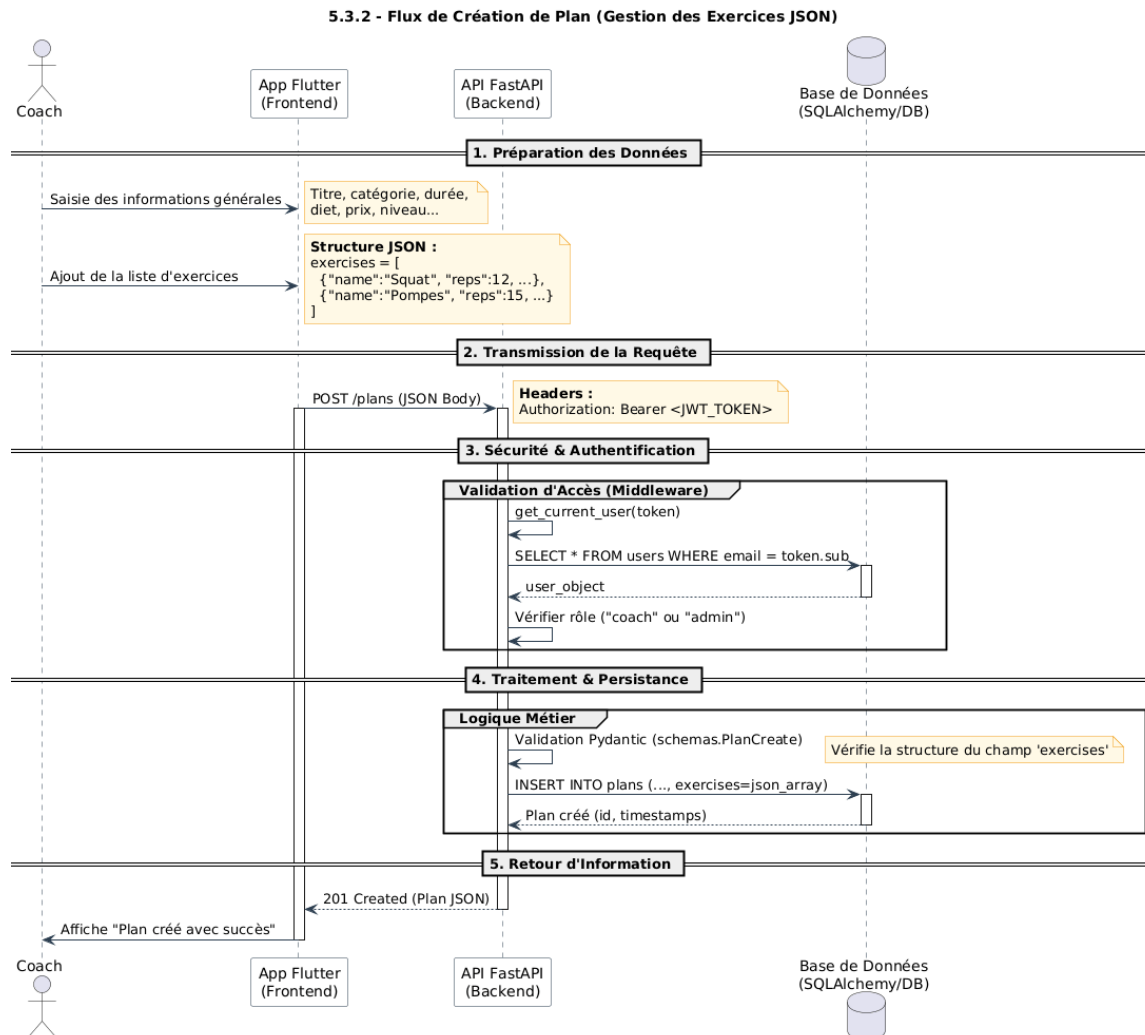


Figure 5.2: Diagramme de séquence : Création d'un plan.

5.4 Diagramme de classe

Le modèle de données s'enrichit de la classe *Plan* et de sa relation avec la classe *User*. Un plan contient des attributs tels que le titre, la description, le prix, et une structure de données pour les exercices.

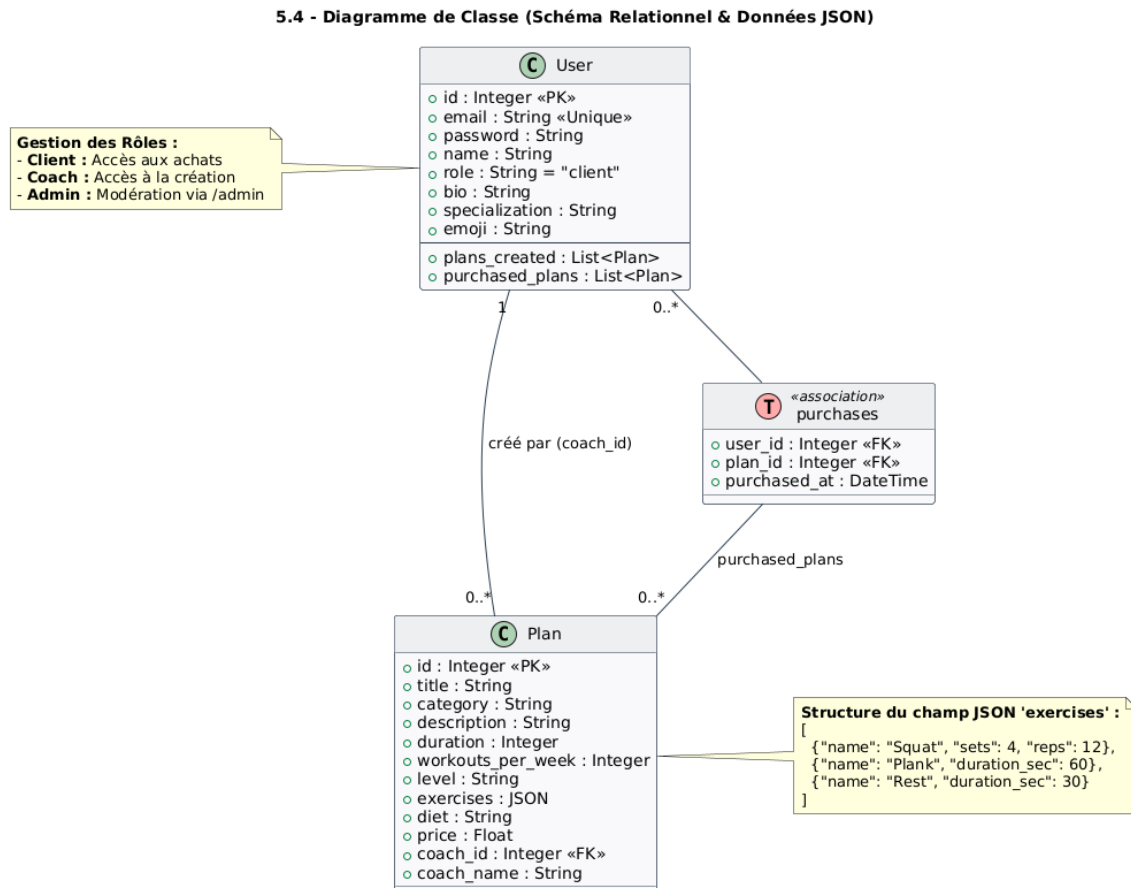


Figure 5.3: Diagramme de classes (Module Plans de Fitness).

5.5 Réalisation

La réalisation de ce sprint met en avant l'aspect visuel du catalogue et la simplicité du formulaire de création.

5.5.1 Interface liste des plans (Catalogue)

L'interface de découverte (*Discover*) affiche les plans sous forme de cartes élégantes, filtrées par catégories (Musculature, Yoga, Cardio). Elle utilise un *ListView* optimisé sous Flutter pour la performance.

5.5.2 Interface de création de plan

Cette interface, réservée aux coachs, permet de saisir les détails du programme. Elle intègre des champs de validation pour s'assurer que chaque plan dispose d'un prix et d'une description cohérente avant l'envoi au serveur.

5.5.3 Interface « gestion des plans »

Le coach dispose d'un espace personnel pour lister ses publications, voir le nombre de souscripteurs et effectuer des mises à jour rapides sur ses tarifs ou le contenu des séances.

5.6 Conclusion

Le Sprint 3 a permis de finaliser la logique fonctionnelle de notre application. En permettant la gestion dynamique des plans d'entraînement, nous avons transformé une simple interface d'authentification en une plateforme de services complète. Ce chapitre clôture la phase de développement intensif. Le chapitre suivant sera consacré à la conclusion générale de ce rapport de stage, dressant le bilan de cette expérience technique et humaine au sein de **IT HOUSE**.

Conclusion et perspectives

Ce rapport résulte d'un travail progressif réalisé dans le cadre de mon stage de perfectionnement pour l'obtention de la licence en **Technologies de l'Informatique** à l'**ISSET Radès**. Durant ce stage, l'entreprise **IT HOUSE** m'a confié la mission de concevoir et de développer une application mobile de suivi fitness. Cette plateforme permet aux utilisateurs de consulter des plans d'entraînement et de suivre leur progression via une architecture moderne combinant **Flutter** et un backend **FastAPI**.

Au-delà des fonctionnalités de base, nous avons intégré un système d'administration complet incluant un tableau de bord web **SQLAdmin** pour la gestion des utilisateurs et des plans, ainsi qu'une interface mobile dédiée aux administrateurs. Ces ajouts assurent une gestion sécurisée et efficace de la plateforme.

Ce projet poursuit un double objectif : faciliter l'accès à un accompagnement sportif de qualité et mettre en pratique les compétences techniques acquises durant ma formation. Le présent rapport retrace les étapes de conception, de développement et de mise en œuvre de la solution, en soulignant les choix fonctionnels et techniques retenus.

L'application développée a permis d'atteindre les objectifs fixés initialement et d'aller au-delà avec les fonctionnalités d'administration. Toutefois, plusieurs pistes d'amélioration peuvent être envisagées afin d'enrichir davantage ses fonctionnalités :

- **Système de recommandation intelligent** : Intégrer un algorithme permettant de proposer des plans d'entraînement personnalisés en fonction de la morphologie et des objectifs de l'utilisateur.
- **Intégration de paiement réel** : Passer d'un achat virtuel à un système de paiement sécurisé (Stripe ou Flouci) pour monétiser les services des coaches.
- **Suivi biométrique** : Connecter l'application aux montres connectées ou aux capteurs de santé pour récupérer en temps réel les calories brûlées et le rythme cardiaque.
- **Interface multilingue** : Ajouter un support pour plusieurs langues afin de rendre la plateforme accessible à un public international.
- **Analytics avancés** : Intégrer des tableaux de bord d'analyse pour suivre les performances des coaches et l'engagement des utilisateurs.

En guise de conclusion, nous avons veillé à donner la meilleure image possible de discipline, de qualités humaines et de compétences techniques auprès du personnel de l'entreprise **IT HOUSE**, dans le but de fournir un travail à la hauteur de leurs attentes et des standards du marché du développement mobile.

Ce projet constitue pour moi une étape cruciale vers une carrière d'ingénieur en développement logiciel, en confirmant mon intérêt pour les architectures mobiles modernes et performantes.

Mounib CHANDOUL

Bibliographie

- [1] Flutter Team. *Flutter Documentation*. Google, 2024.
<https://flutter.dev/docs>
- [2] Sebastián Ramirez. *FastAPI Documentation*. 2024.
<https://fastapi.tiangolo.com/>
- [3] SQLAlchemy Team. *SQLAlchemy Documentation*. 2024.
<https://www.sqlalchemy.org/>
- [4] Amina laee. *SQLAdmin Documentation*. 2024.
<https://aminalaee.dev/sqladmin/>
- [5] Jones, M., Bradley, J., Sakimura, N. *JSON Web Token (JWT) Specification*. RFC 7519, IETF, 2015.
- [6] Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine, 2000.
- [7] Google. *Material Design Guidelines*. 2024.
<https://material.io/design>
- [8] Google. *Dart Programming Language*. 2024.
<https://dart.dev/>
- [9] Python Software Foundation. *Python Documentation*. 2024.
<https://www.python.org/doc/>

Glossaire

API Application Programming Interface : Interface de programmation permettant à des applications de communiquer entre elles.

Backend Partie serveur d'une application, responsable de la logique métier et de la gestion des données.

CRUD Create, Read, Update, Delete : Les quatre opérations de base pour gérer les données.

Frontend Interface utilisateur d'une application, visible par l'utilisateur final.

HTTP HyperText Transfer Protocol : Protocole de communication pour le web.

JWT JSON Web Token : Standard pour créer des jetons d'accès sécurisés.

ORM Object-Relational Mapping : Technique pour convertir des données entre systèmes incompatibles.

REST Representational State Transfer : Style architectural pour les services web.

SQL Structured Query Language : Langage pour gérer les bases de données relationnelles.

UI/UX User Interface/User Experience : Interface utilisateur et expérience utilisateur.

Annexes

6. Code Source Principal

6.1 Modèle Utilisateur (Backend)

```
1 class User(Base):
2     __tablename__ = "users"
3     id = Column(Integer, primary_key=True, index=True)
4     email = Column(String, unique=True, index=True)
5     password = Column(String)
6     name = Column(String, index=True, nullable=False)
7     role = Column(String, default="client")
8     bio = Column(String, nullable=True, default="")
9     specialization = Column(String, nullable=True, default="")
10    emoji = Column(String, nullable=True, default=" ")
11
12    plans_created = relationship("Plan", back_populates="coach", cascade
13    = "all, delete-orphan")
14    purchased_plans = relationship("Plan", secondary=purchases,
15    back_populates="buyers")
```

Listing 6.1: Modèle SQLAlchemy pour les utilisateurs

6.2 Endpoint d'Authentification

```
1 @router.post("/login")
2 def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session
3   = Depends(get_db)):
4     user = db.query(models.User).filter(models.User.email == form_data.
5     username).first()
6     if not user or user.password != form_data.password:
7         raise HTTPException(status_code=400, detail="Invalid credentials
8         ")
9
10    token = create_access_token(data={"sub": user.email})
11    return {
12        "access_token": token,
13        "token_type": "bearer",
14        "role": user.role,
15        "name": user.name,
16        "id": user.id,
17        "bio": getattr(user, "bio", ""),
18        "specialization": getattr(user, "specialization", ""),
19        "emoji": getattr(user, "emoji", " ")}
```

```
17 }
```

Listing 6.2: Route d'authentification FastAPI

6.3 Configuration SQLAlchemy

```
1 class AdminAuth(AuthenticationBackend):
2     async def login(self, request: Request) -> bool:
3         form = await request.form()
4         email = form.get("username")
5         password = form.get("password")
6         db = models.SessionLocal()
7         try:
8             user = db.query(models.User).filter(models.User.email ==
email).first()
9             if user and pwd_context.verify(password, user.password)
10                 and user.role == "admin":
11                 return True
12         finally:
13             db.close()
14         return False
15
16     async def authenticate(self, request: Request) -> bool:
17         return "user" in request.session
18
19 admin = Admin(api, models.engine, authentication_backend=AdminAuth(
secret_key="..."))
20
21 class UserAdmin(ModelView, model=models.User):
22     column_list = [models.User.id, models.User.email, models.User.name,
models.User.role]
23
24 class PlanAdmin(ModelView, model=models.Plan):
25     column_list = [models.Plan.id, models.Plan.title, models.Plan.
category, models.Plan.coach_name, models.Plan.price]
26
27 admin.add_view(UserAdmin)
28 admin.add_view(PlanAdmin)
```

Listing 6.3: Configuration du tableau de bord d'administration

6.4 Interface Utilisateur Flutter

```
1 class LoginPage extends StatefulWidget {
```



```

2  const LoginPage({super.key});
3
4  @override
5  State<LoginPage> createState() => _LoginPageState();
6  }
7
8  class _LoginPageState extends State<LoginPage> {
9    final _emailController = TextEditingController();
10   final _passwordController = TextEditingController();
11
12   Future<void> _handleLogin() async {
13     final email = _emailController.text.trim();
14     final password = _passwordController.text;
15
16     final response = await http.post(
17       Uri.parse("${Config.apiUrl}/auth/login"),
18       headers: {"Content-Type": "application/x-www-form-urlencoded"},
19       body: {"username": email, "password": password},
20     );
21
22     if (response.statusCode == 200) {
23       final data = jsonDecode(response.body);
24       final prefs = await SharedPreferences.getInstance();
25       await prefs.setString('jwt_token', data['access_token']);
26       await prefs.setString('user_role', data['role']);
27       await prefs.setString('username', data['name']);
28
29       Navigator.pushReplacement(
30         context,
31         MaterialPageRoute(builder: (_) => MainShell(user_name: data['
32         name'])),
33       );
34     }
35   }
36   // ... reste du code

```

Listing 6.4: Écran de connexion Flutter

7. Diagrammes Supplémentaires

7.1 Diagramme de Gantt

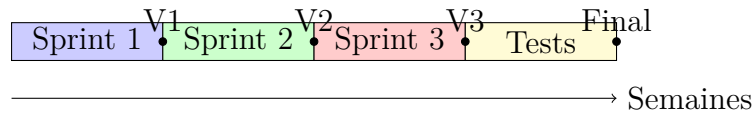


Figure 7.1: Diagramme de Gantt du projet

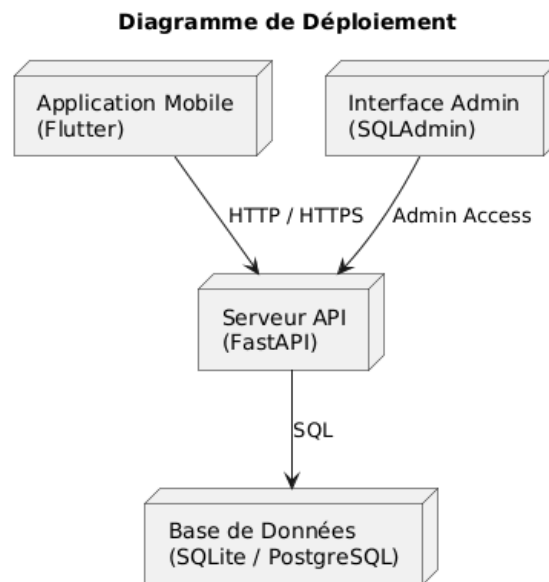


Figure 7.2: Diagramme de déploiement.

8. Tests et Validation

8.1 Tests Unitaires

```
1 def test_create_user():
2     response = client.post("/auth/register", json={
3         "email": "test@example.com",
4         "password": "test123",
5         "name": "Test User"
6     })
7     assert response.status_code == 200
8     data = response.json()
9     assert data["email"] == "test@example.com"
10    assert data["name"] == "Test User"
```

Listing 8.1: Exemple de test FastAPI

8.2 Tests d'Interface

- **Test de connexion** : Vérification de l'authentification avec différents rôles
- **Test de création de plan** : Validation de la soumission de formulaires complexes
- **Test d'administration** : Contrôle d'accès aux fonctionnalités admin
- **Test de performance** : Mesure des temps de réponse API

9. Guide d'Installation et Déploiement

9.1 Prérequis

- Python 3.8+
- Flutter 3.0+
- SQLite ou PostgreSQL
- Git

9.2 Installation Backend

```
1 # Cloner le repository
2 git clone <repository-url>
3 cd fitness_backed
4
5 # Créer l'environnement virtuel
6 python -m venv .venv
7 source .venv/bin/activate
8
9 # Installer les dépendances
10 pip install -r requirements.txt
11
12 # Lancer le serveur
13 fastapi dev me.py
```

Listing 9.1: Installation du backend

9.3 Installation Frontend

```
1 # Cloner le repository
2 cd fitness
3
4 # Installer les dépendances
5 flutter pub get
6
7 # Lancer l'application
8 flutter run
```

Listing 9.2: Installation du frontend

9.4 Configuration Production

- Configurer une base de données PostgreSQL
- Définir les variables d'environnement (SECRET_KEY, DATABASE_URL)
- Configurer un serveur web (Nginx + Gunicorn)
- Activer HTTPS avec Let's Encrypt